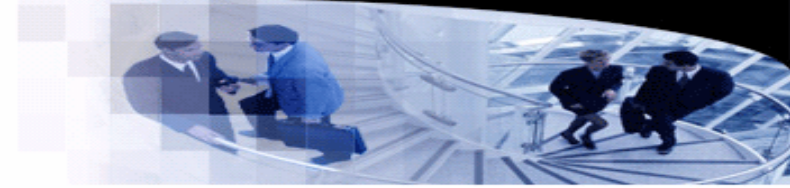




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

Lecture 30. Register Allocation

袁梦霆 (YUAN Mengting)

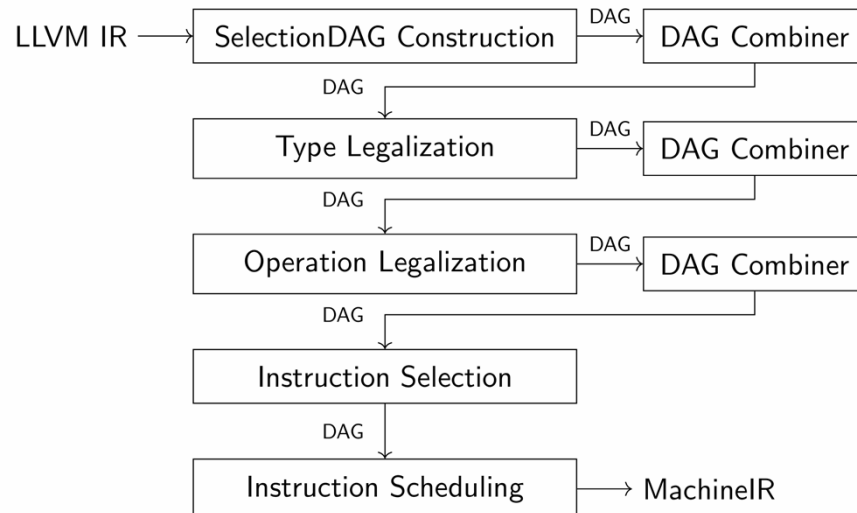
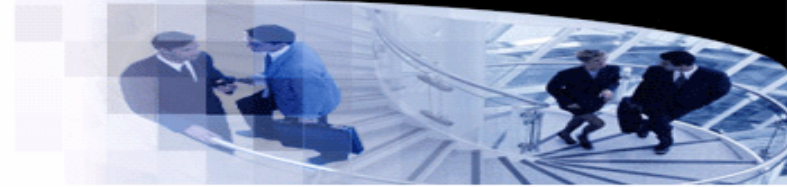
ymt@whu.edu.cn

School of Computer Science, Wuhan University

Register Allocation

- What have we learned so far?

- The source program is translated into machine IR.
- A sequential intermediate representation.
- Types and operations are aligned to the target machine.
- The only problem is that machine IR uses virtual registers.



Machine code for function _Z3foov: NoPHIs, TracksLiveness, TiedOpsRewritten

```
0B bb.0.entry:
16B %0:gpr = LUI target-flags(riscv-hi) @b
32B %1:gpr = LW killed %0:gpr, target-flags(riscv-lo) @b :: (dereferenceable load (s32) from @b)
48B %2:gpr = LUI target-flags(riscv-hi) @c
64B %3:gpr = LW killed %2:gpr, target-flags(riscv-lo) @c :: (dereferenceable load (s32) from @c)
80B %4:gpr = SUB killed %1:gpr, killed %3:gpr
96B %5:gpr = LUI target-flags(riscv-hi) @a
112B SW killed %4:gpr, killed %5:gpr, target-flags(riscv-lo) @a :: (store (s32) into @a)
128B PseudoRET
```

Register Allocation

- **Problem statement**

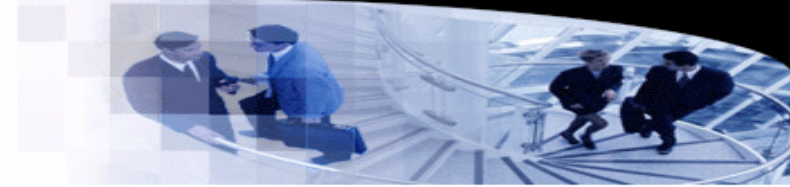
- Given an intermediate representation that uses a set of virtual registers, find a map from virtual registers to physical registers.

- **Spilling**

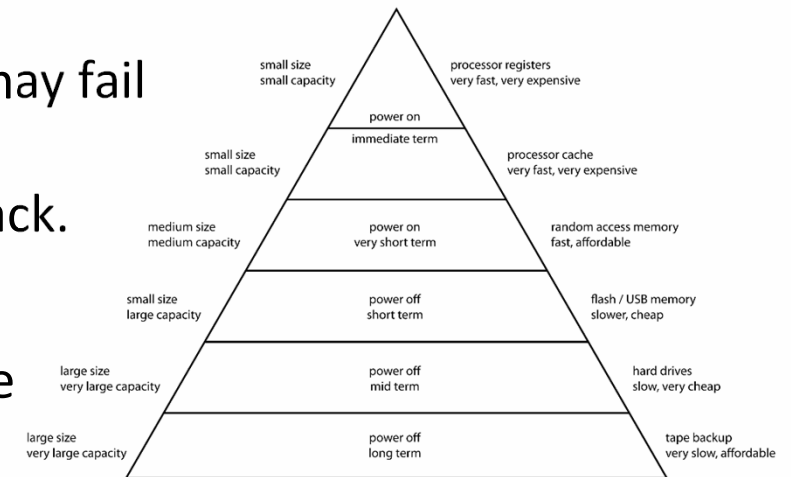
- As the number of virtual registers is not limited, it may fail to map all virtual registers to physical registers.
- Some virtual registers have to be “spilled” to the stack.

- **Requirements**

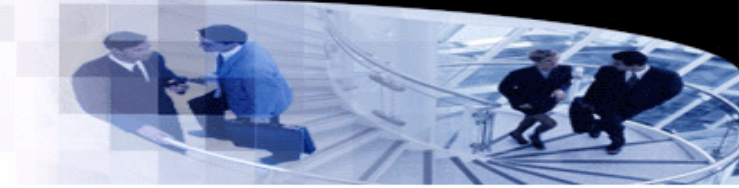
- Produce correct codes when the virtual registers are replaced with physical registers.
- Minimize the overhead of introduced loads and stores.
- Minimize the number of spilled registers.



Computer Memory Hierarchy

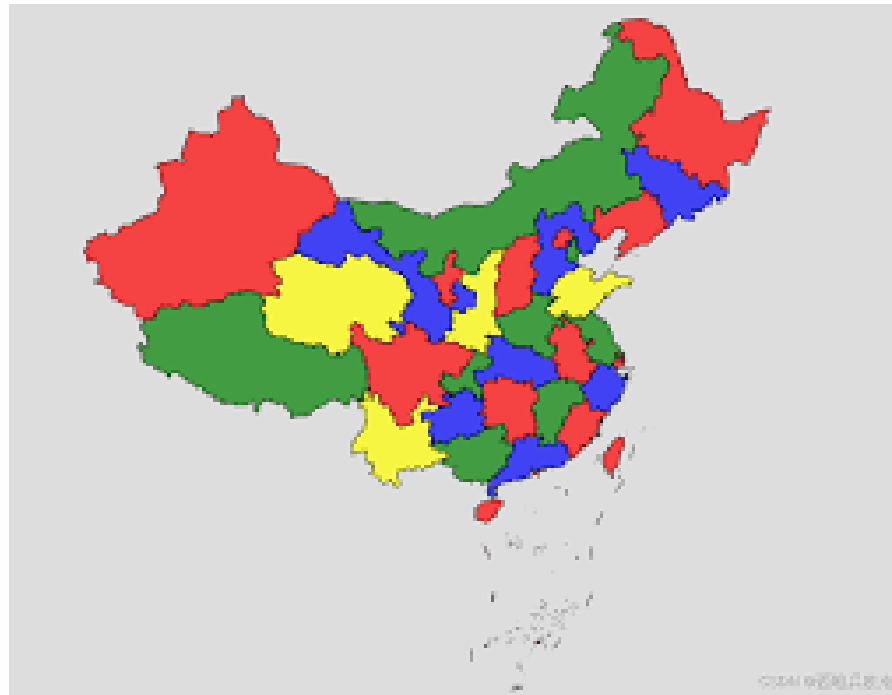


Register Allocation via Graph Coloring

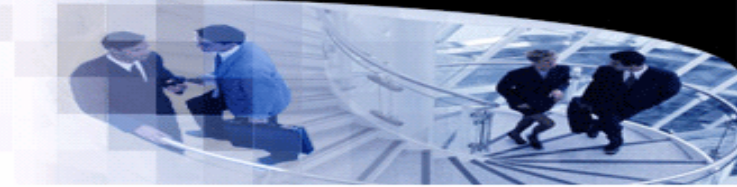


- Graph coloring

- Assign labels (colors) to vertices such that no two adjacent vertices have the same color.



Register Allocation via Graph Coloring



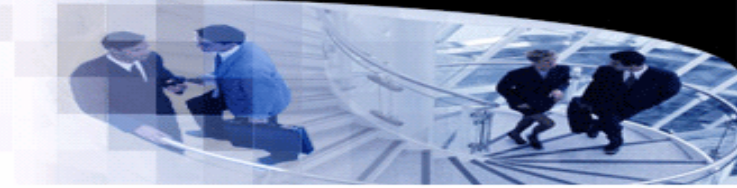
- **Idea**

- Many allocation problems are hard to solve.
- Register allocation is NP-complete.
- We convert register allocation problems into graph coloring problems.
- Vertices are used to represent virtual registers.
- Each color is considered as a physical registers.
- Two adjacent vertices do not share the same color (physical register).

- **Issue**

- Graph coloring is also NP-complete.
- A heuristic algorithm is proposed to solve graph coloring problems.
- Chaitin's algorithm.

Register Allocation via Graph Coloring



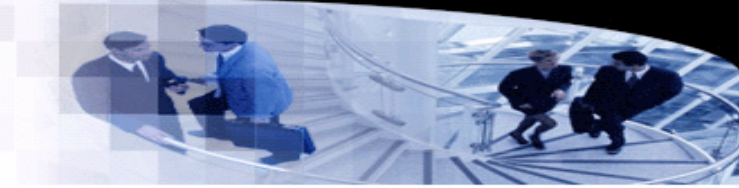
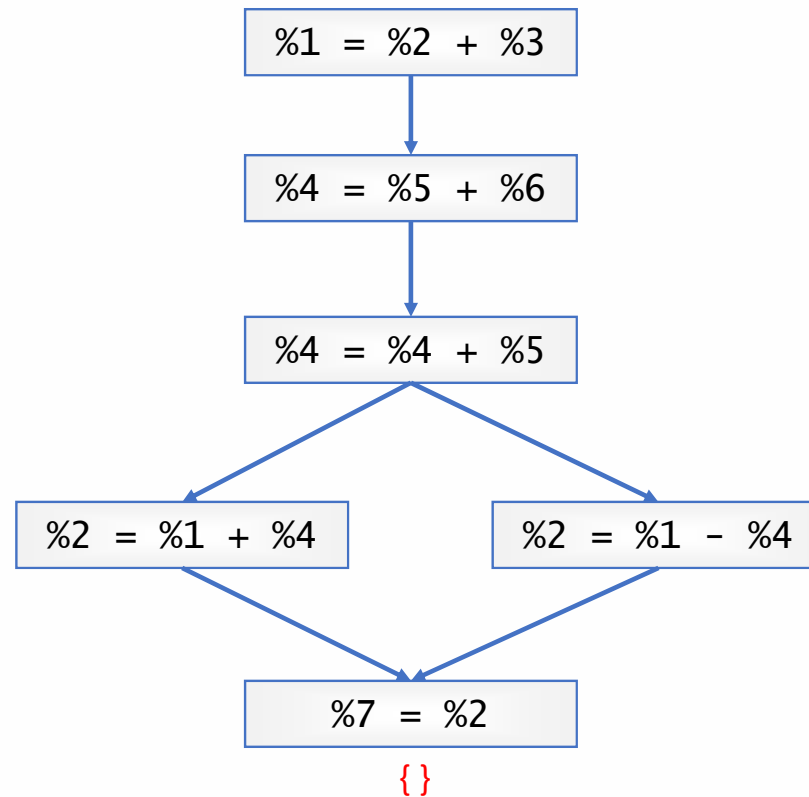
- Sketch of algorithm

- Perform liveness analysis.
- Construct interference graph G using the result of liveness analysis.
- Assign k colors to n vertices of the graph.
- Heuristic:
 - G is k -colorable iff $G - \{v\}$ is k -colorable, where v is a vertex with $k - 1$ edges.
- Select a vertex v with fewer than k edges.
- Recursively color $G - \{v\}$.
- Assign a valid color to v in G .

Register Allocation via Graph Coloring

- Example

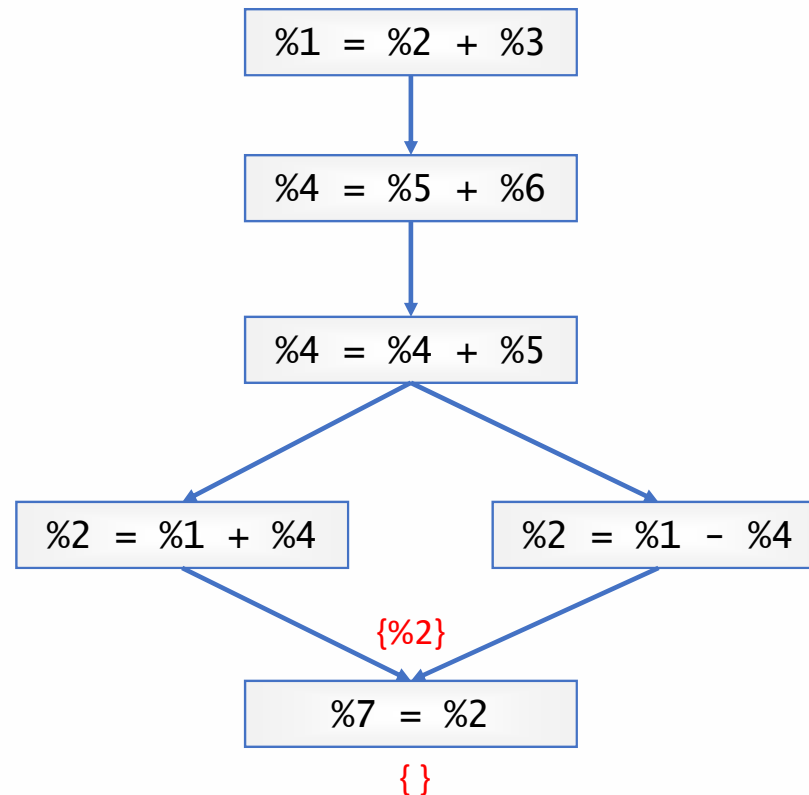
- Liveness analysis



Register Allocation via Graph Coloring

- Example

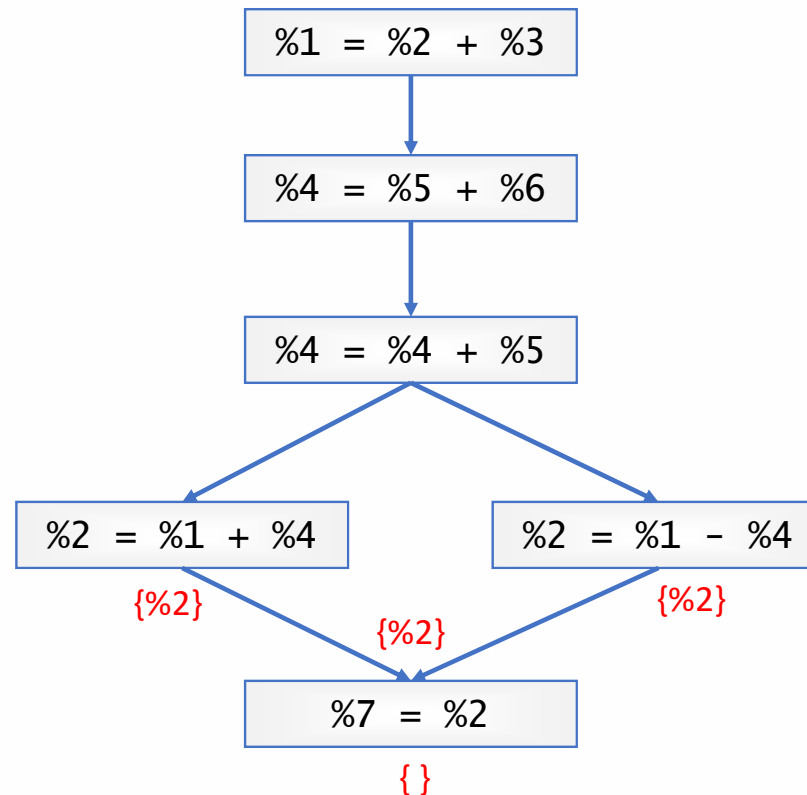
- Liveness analysis



Register Allocation via Graph Coloring

- Example

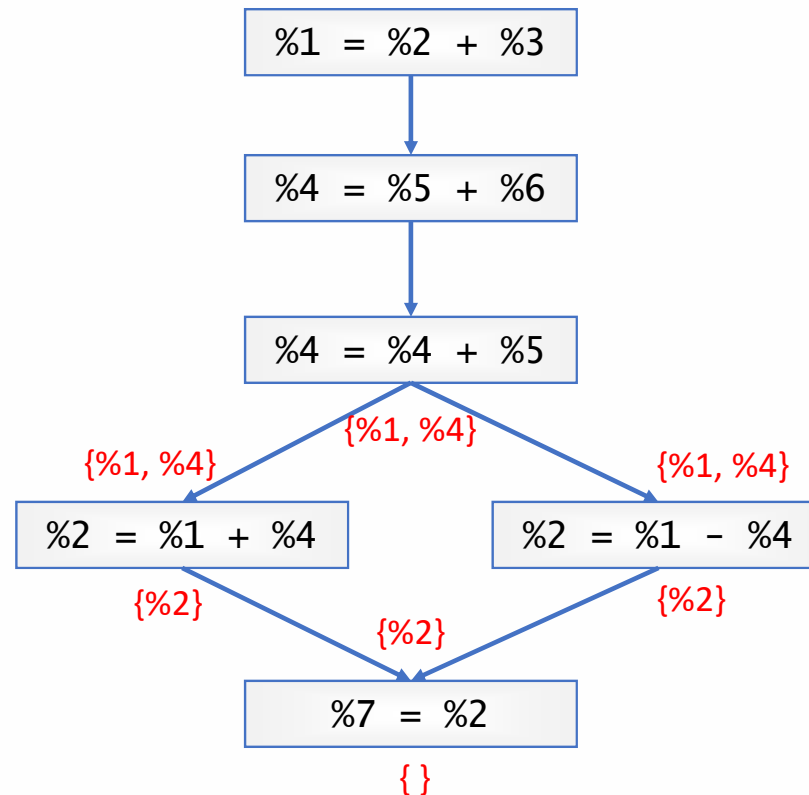
- Liveness analysis



Register Allocation via Graph Coloring

- Example

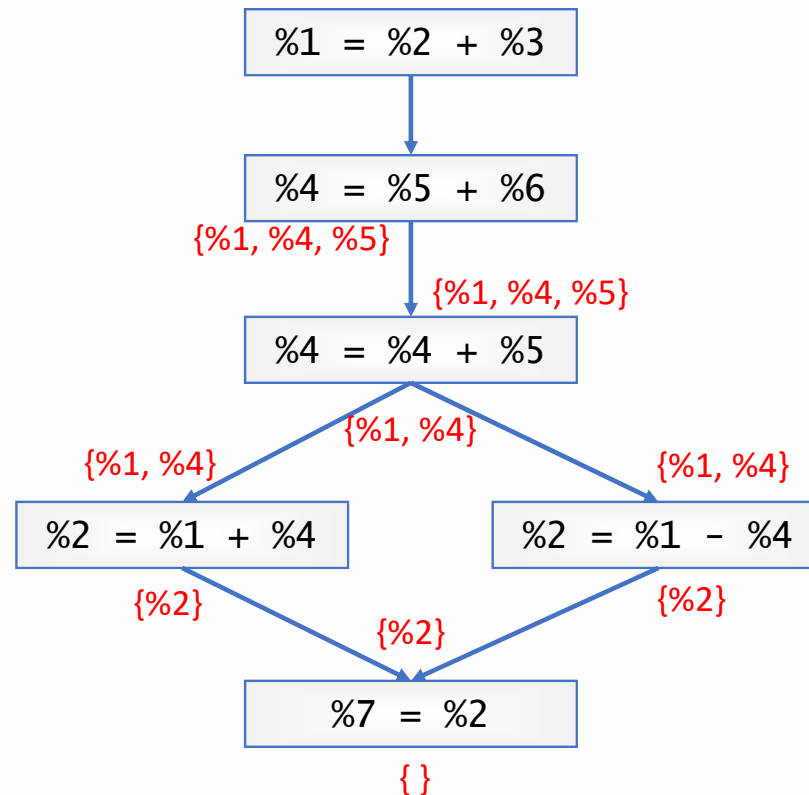
- Liveness analysis



Register Allocation via Graph Coloring

- Example

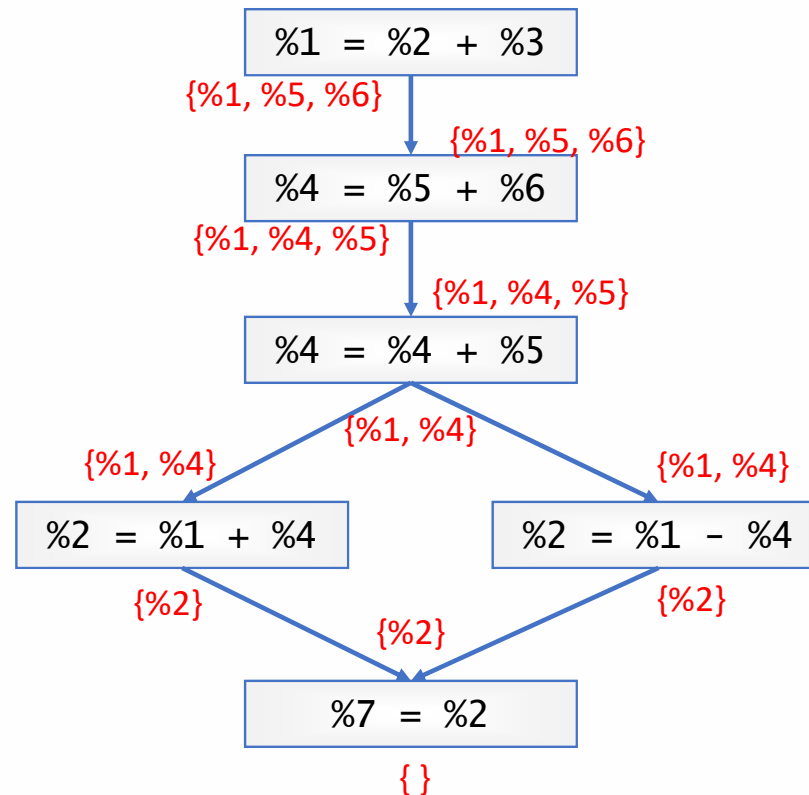
- Liveness analysis



Register Allocation via Graph Coloring

- Example

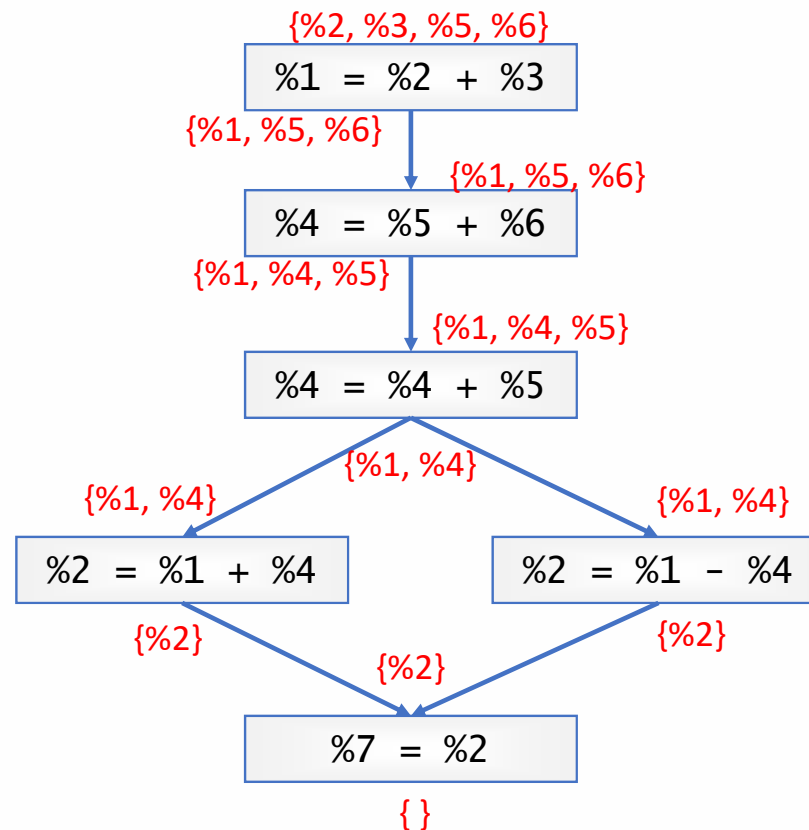
- Liveness analysis



Register Allocation via Graph Coloring

- Example

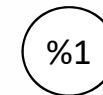
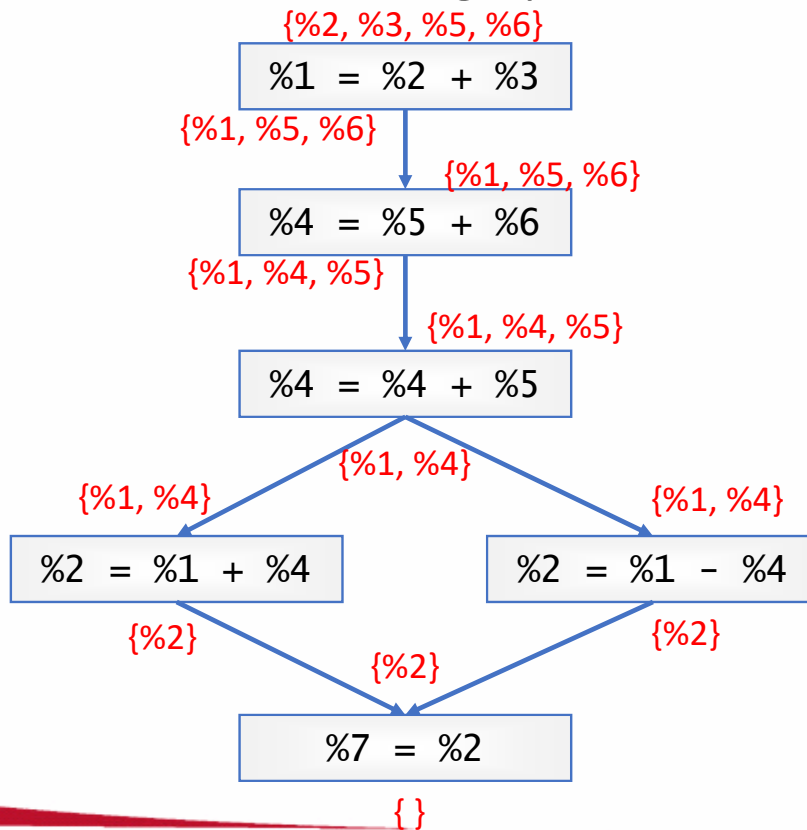
- Liveness analysis



Register Allocation via Graph Coloring

- Example

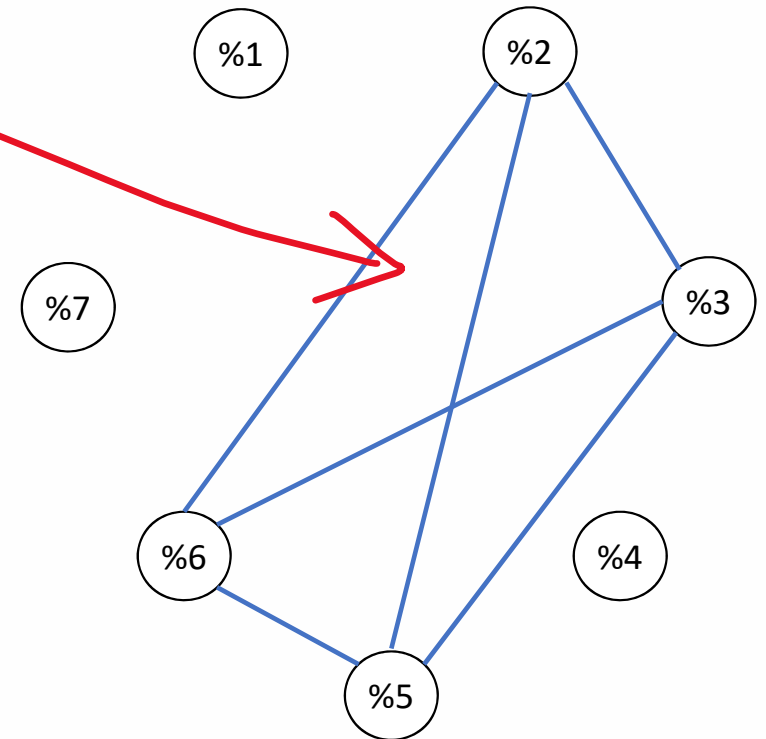
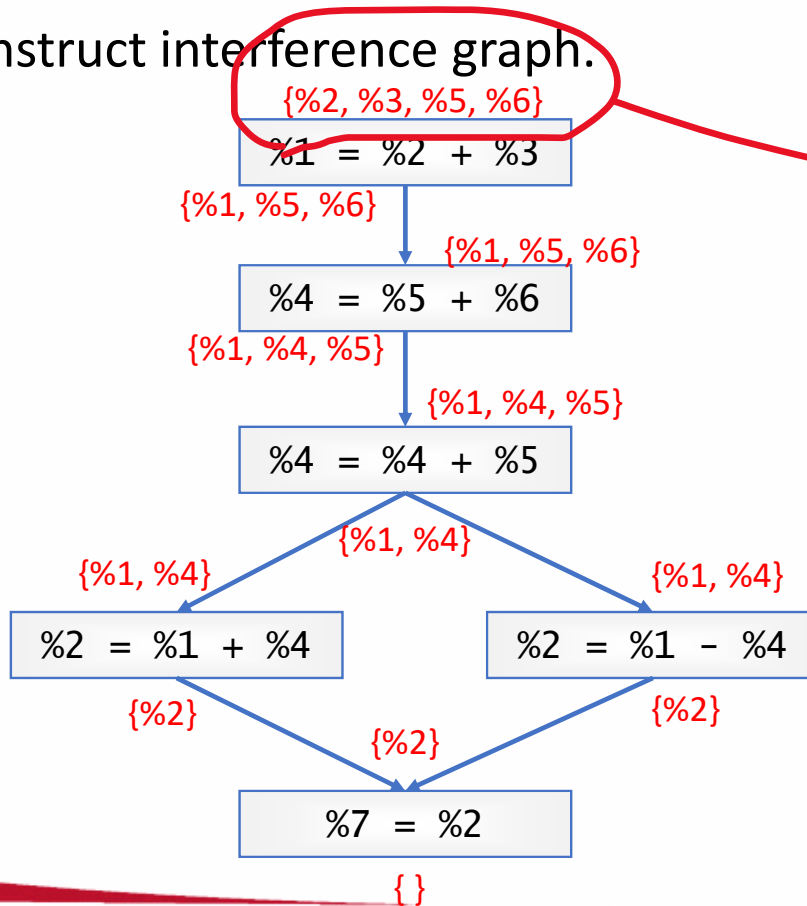
- Construct interference graph.



Register Allocation via Graph Coloring

- Example

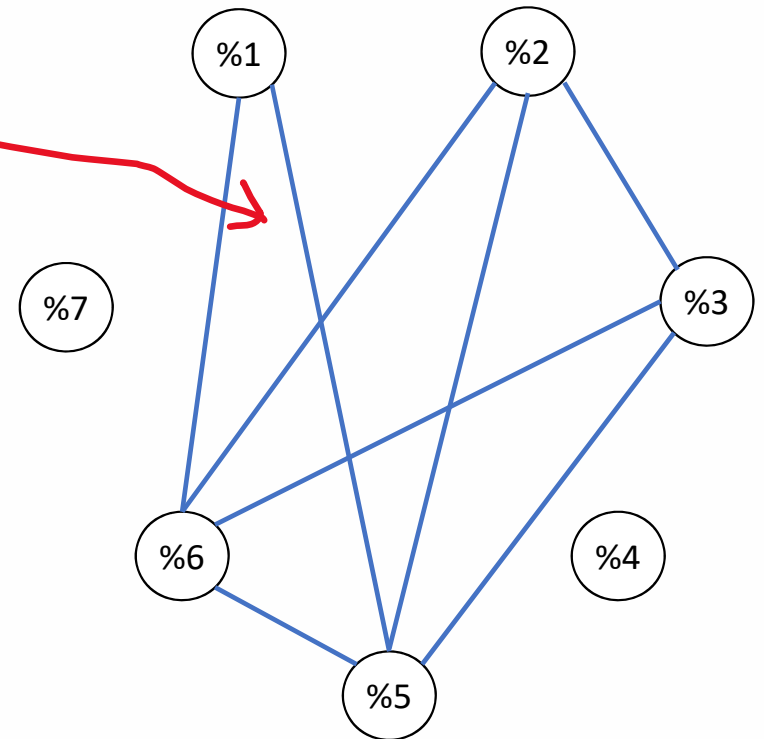
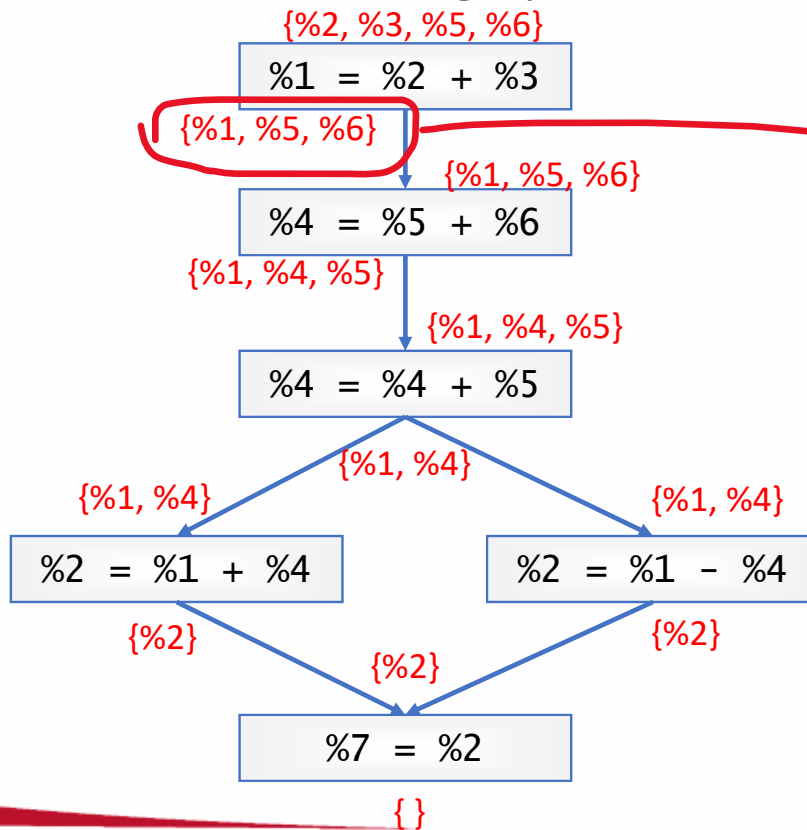
- Construct interference graph.



Register Allocation via Graph Coloring

- Example

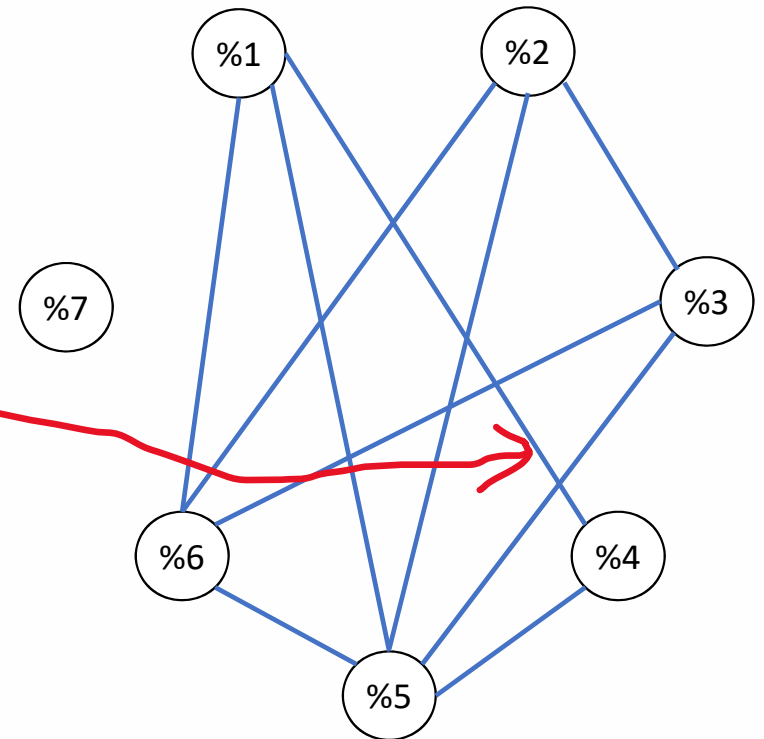
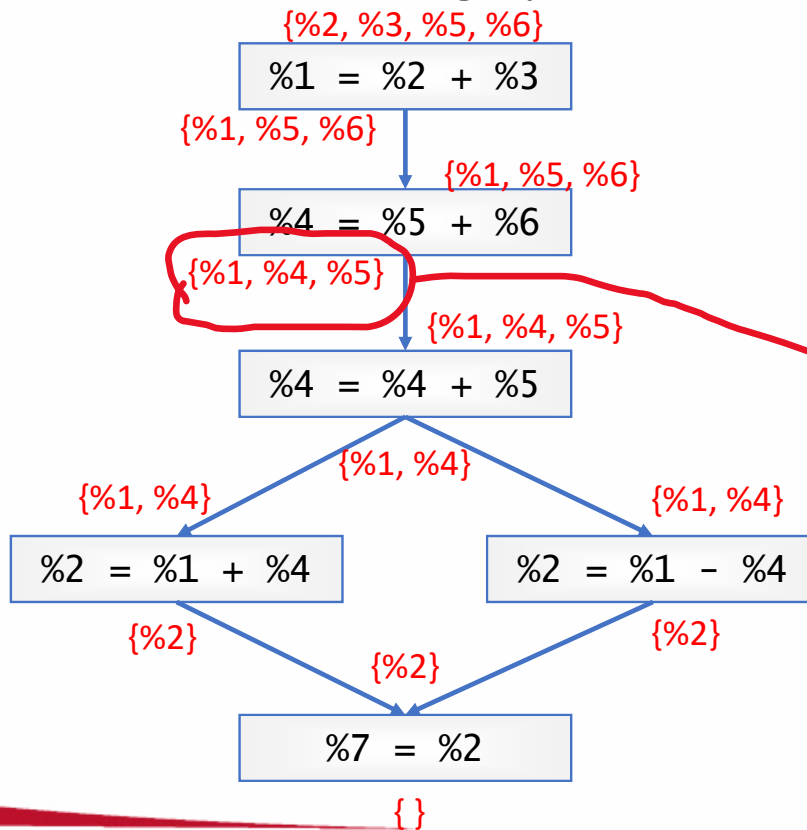
- Construct interference graph.



Register Allocation via Graph Coloring

- Example

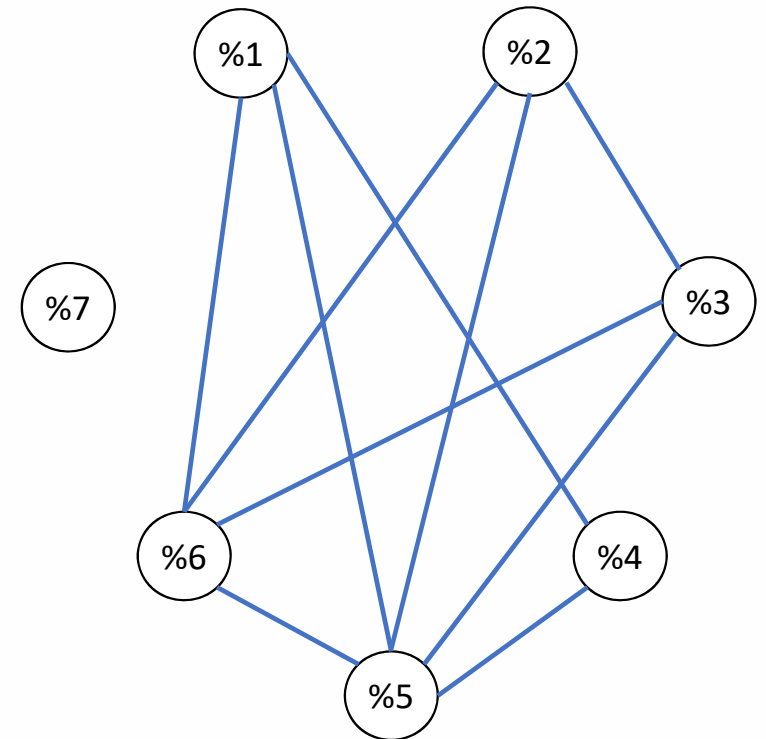
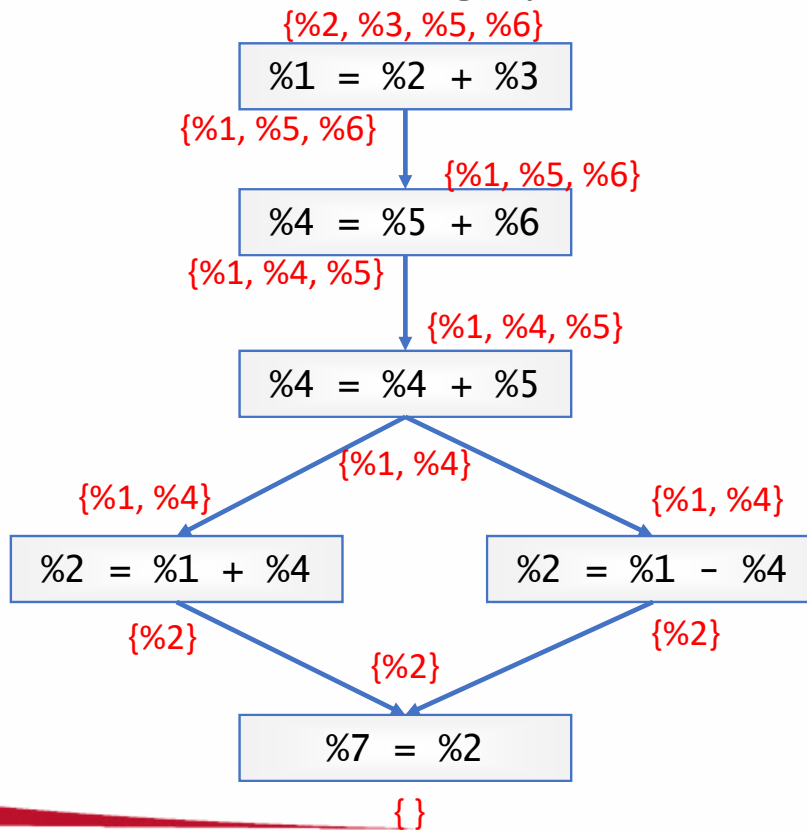
- Construct interference graph.



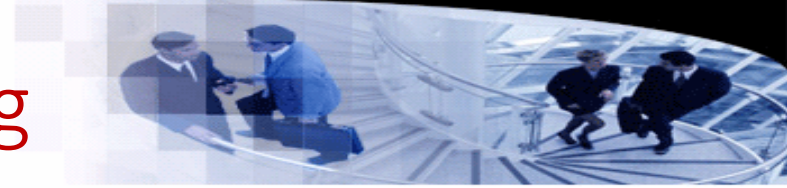
Register Allocation via Graph Coloring

- Example

- Construct interference graph.



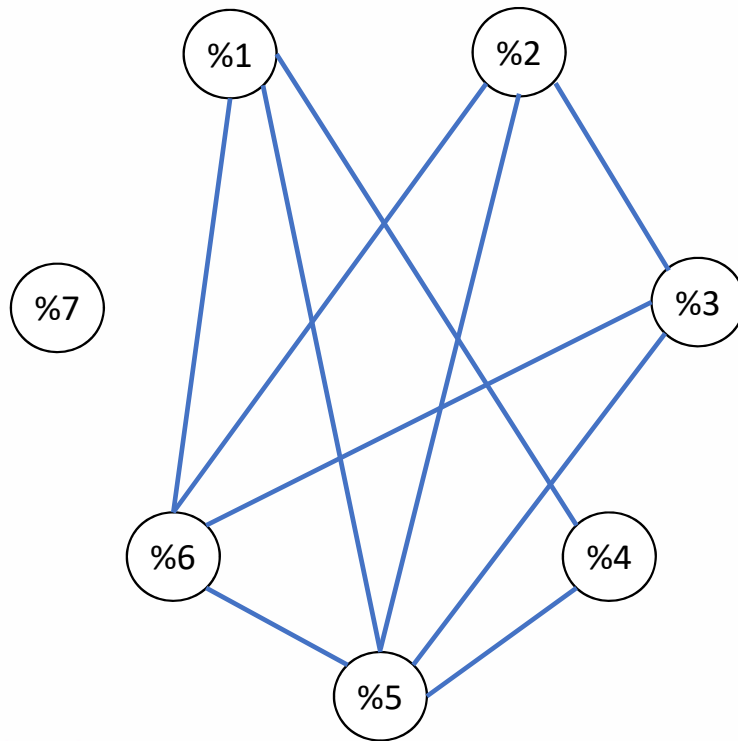
Register Allocation via Graph Coloring



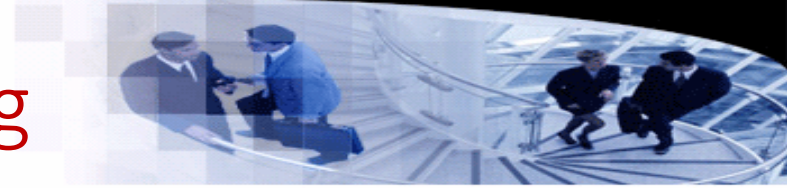
- Example

- Remove vertices.

Removed vertices

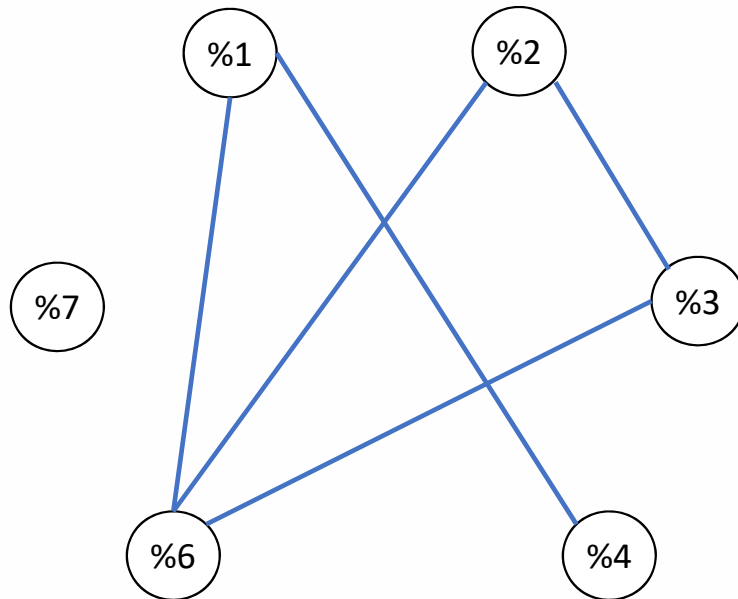


Register Allocation via Graph Coloring



- Example

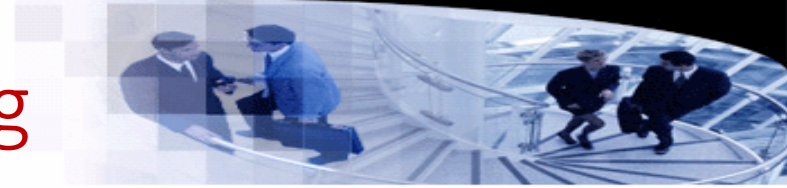
- Remove vertices.



Removed vertices

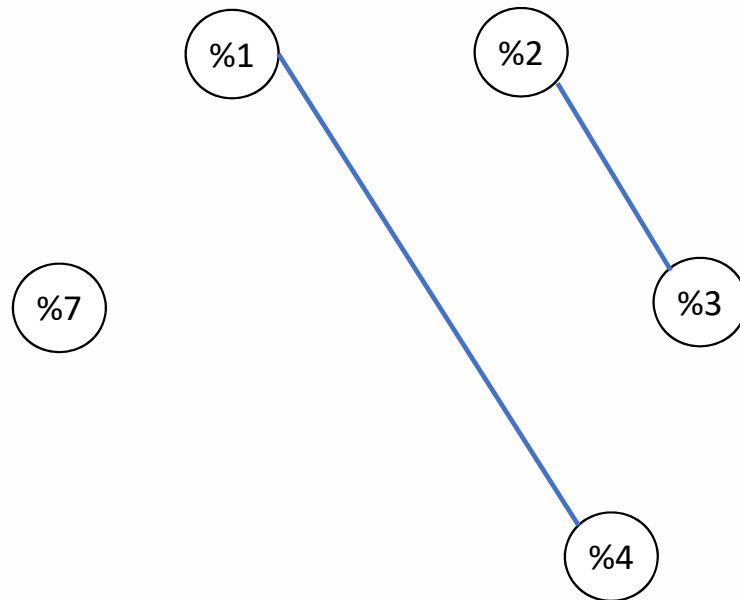


Register Allocation via Graph Coloring



- Example

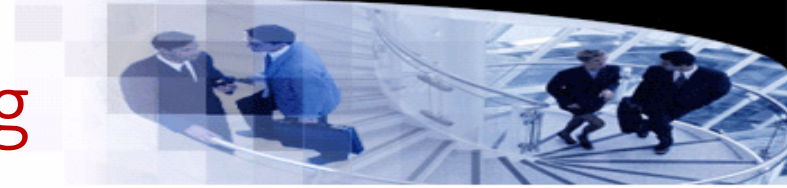
- Remove vertices.



Removed vertices

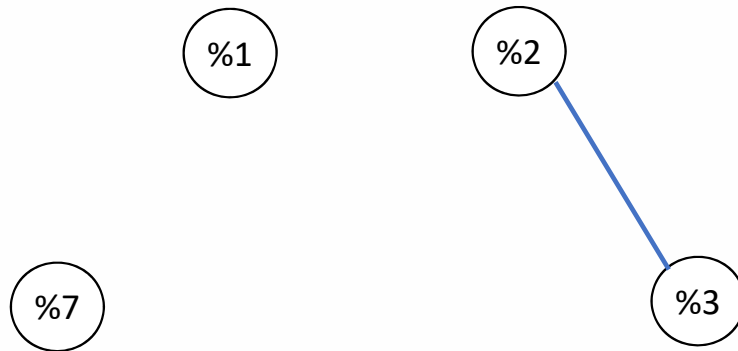


Register Allocation via Graph Coloring



- Example

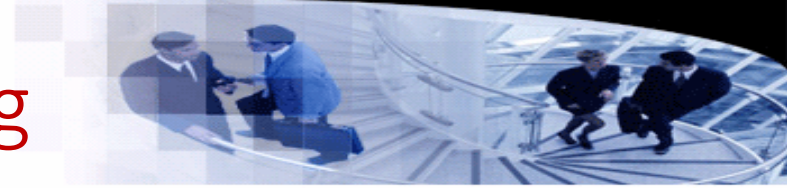
- Remove vertices.



Removed vertices

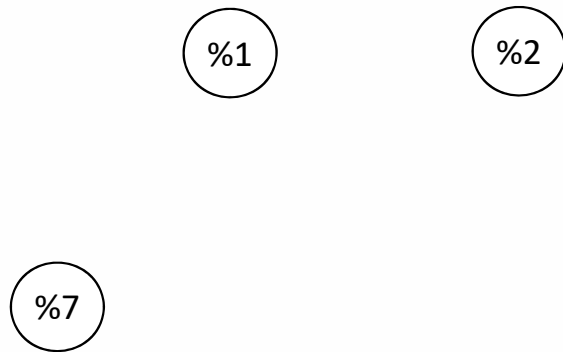


Register Allocation via Graph Coloring



- Example

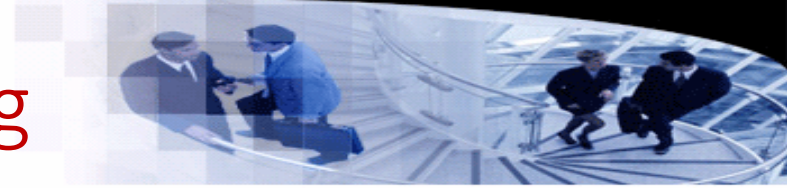
- Remove vertices.



Removed vertices



Register Allocation via Graph Coloring



- Example

- Remove vertices.

Removed vertices

%5

%6

%4

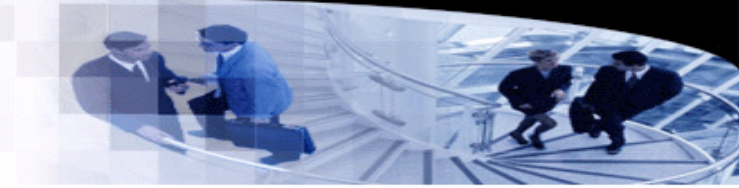
%3

%2

%1

%7

Register Allocation via Graph Coloring



- **Example**

- Start allocation via restoring vertices.
- 6 registers.

Removed vertices

%5

%6

%4

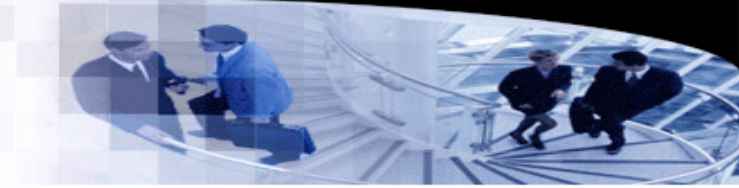
%3

%2

%1

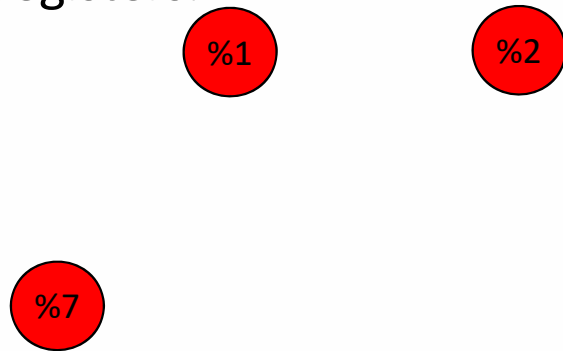
%7

Register Allocation via Graph Coloring



- Example

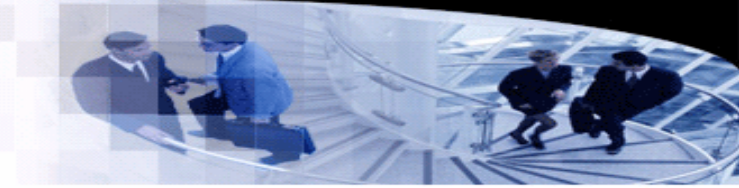
- Start allocation via restoring vertices.
- 6 registers.



Removed vertices

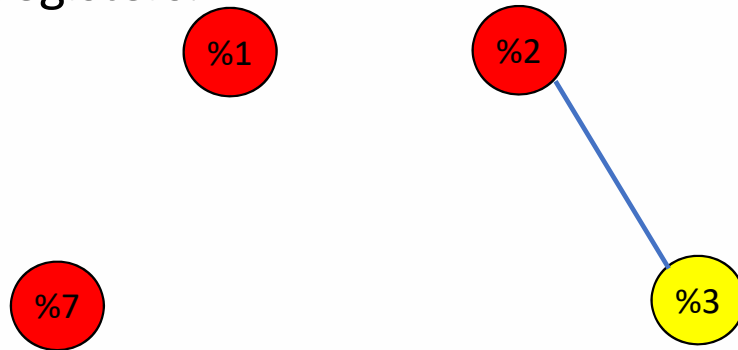


Register Allocation via Graph Coloring



- **Example**

- Start allocation via restoring vertices.
- 6 registers.



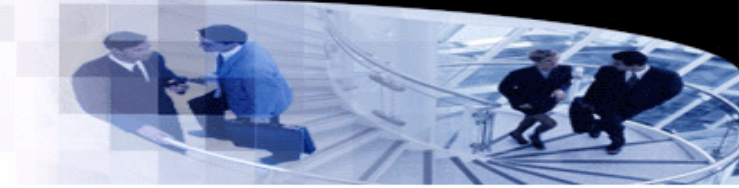
Removed vertices

%5

%6

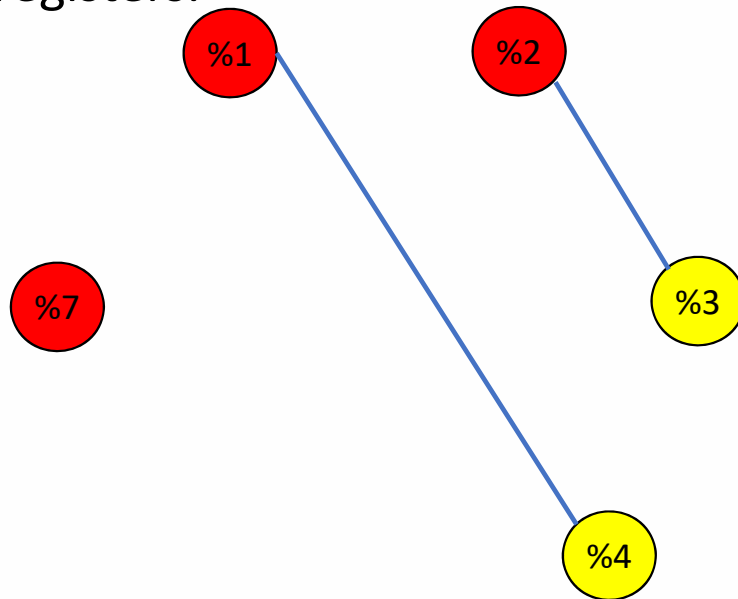
%4

Register Allocation via Graph Coloring



- **Example**

- Start allocation via restoring vertices.
- 6 registers.

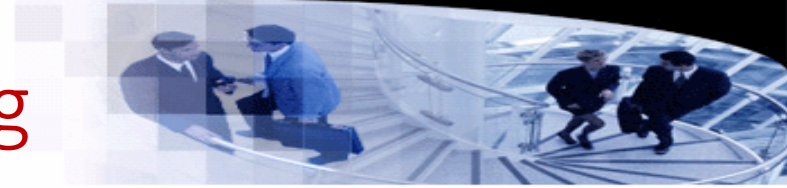


Removed vertices

%5

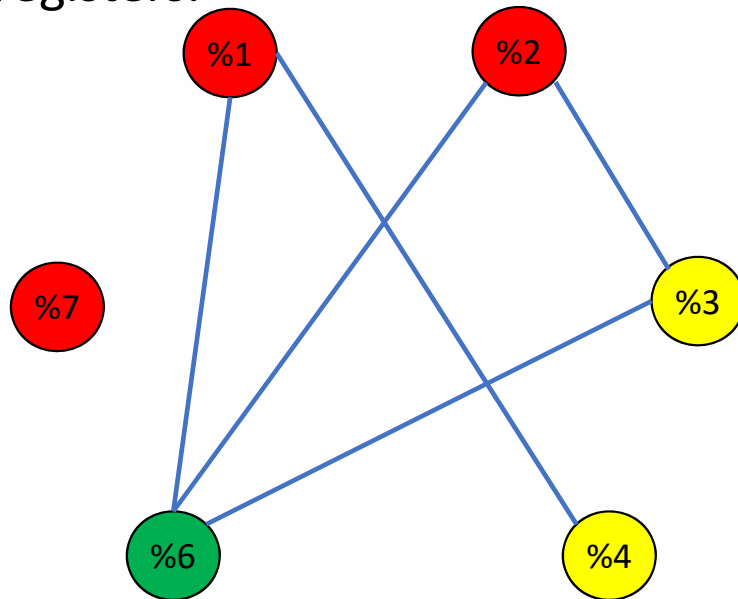
%6

Register Allocation via Graph Coloring



- **Example**

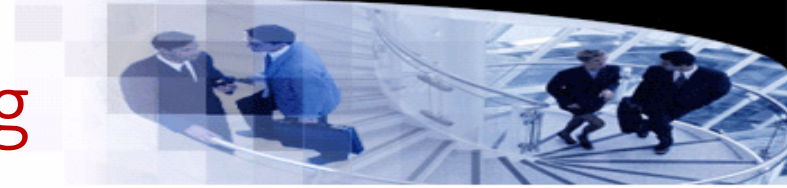
- Start allocation via restoring vertices.
- 6 registers.



Removed vertices



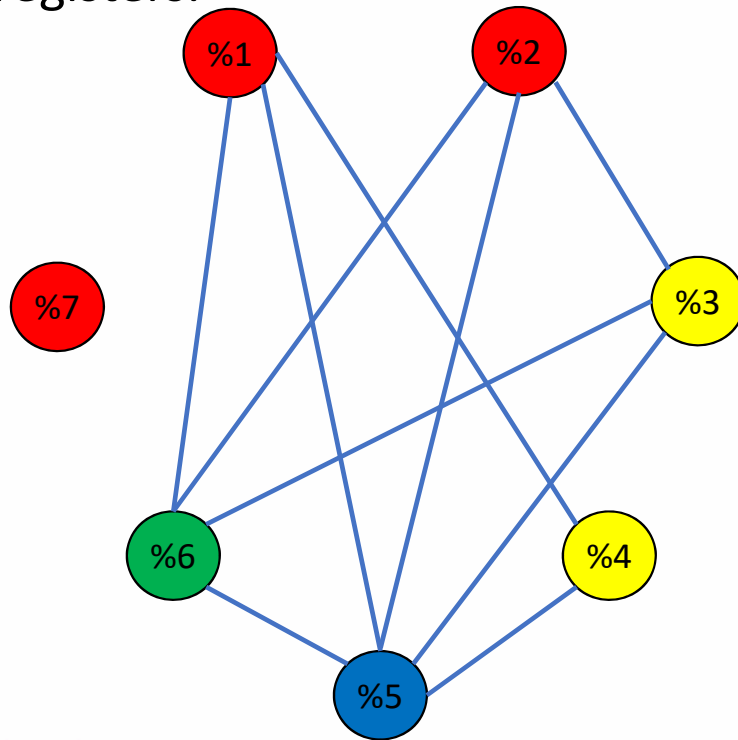
Register Allocation via Graph Coloring



- Example

- Start allocation via restoring vertices.
- 6 registers.

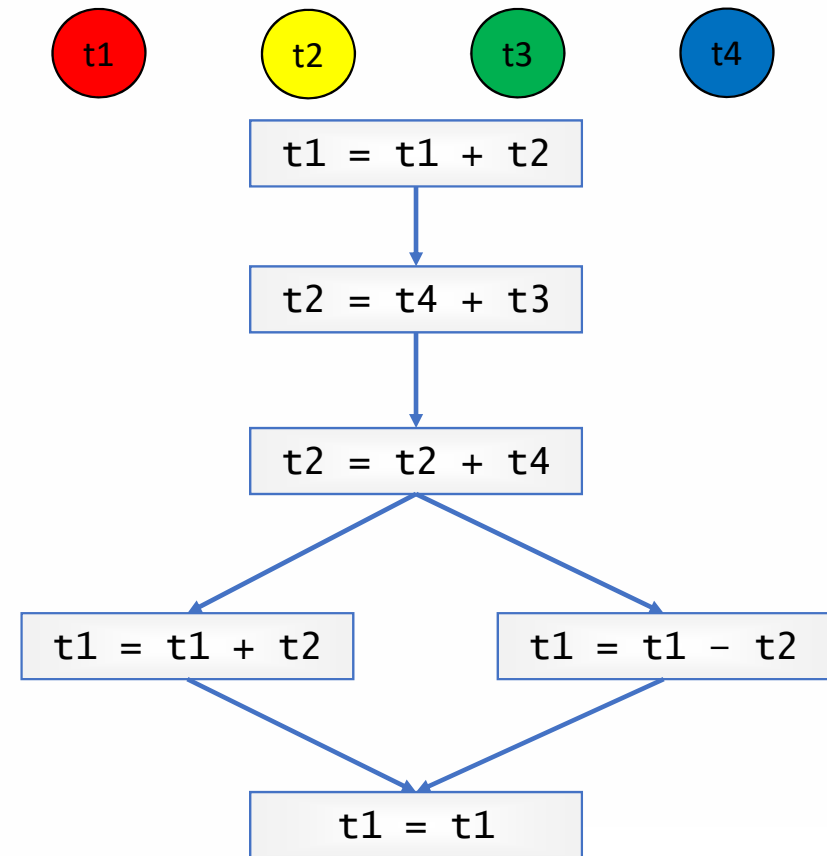
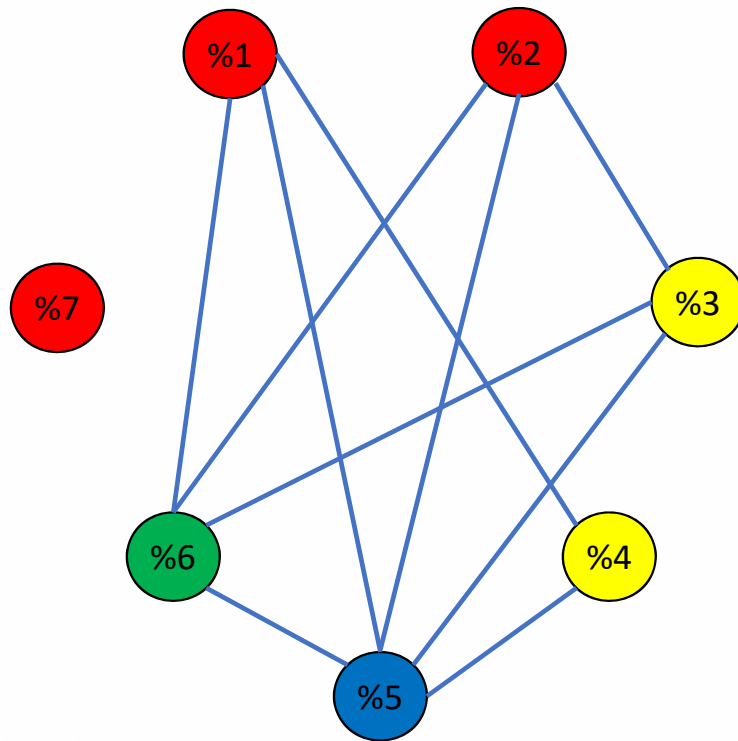
Removed vertices



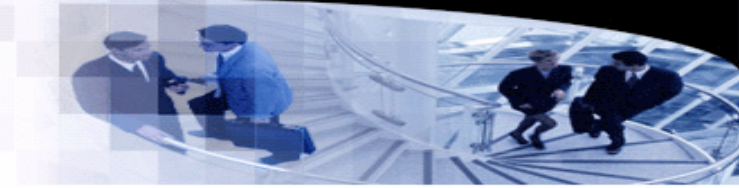
Register Allocation via Graph Coloring

- Example

- Rewrite the program.

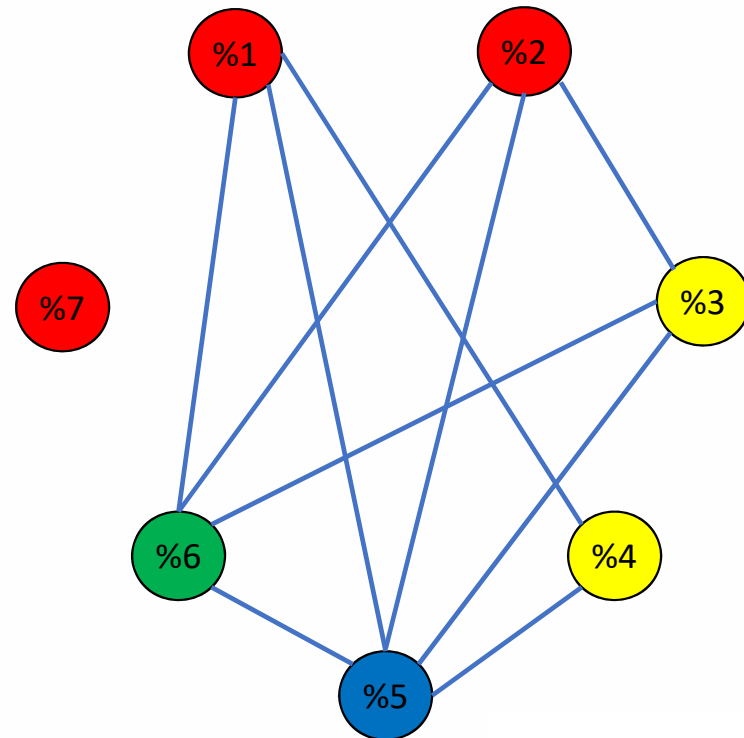


Register Allocation via Graph Coloring

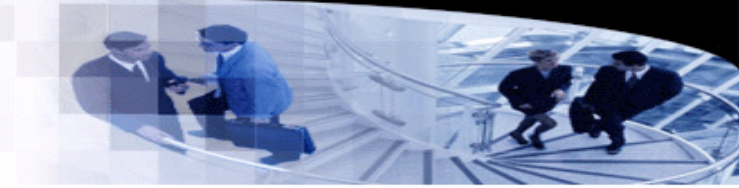


- **Example**

- 4 colors (physical registers) are allocated.
- Note that %5 has 5 edges.
- The graph is still 4-colorable.

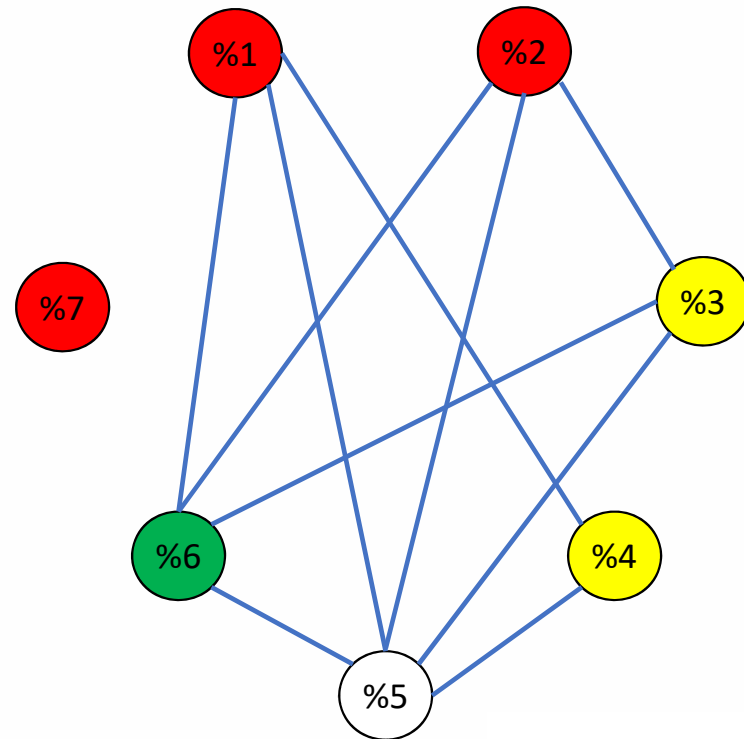


Register Allocation via Graph Coloring



- **Spill**

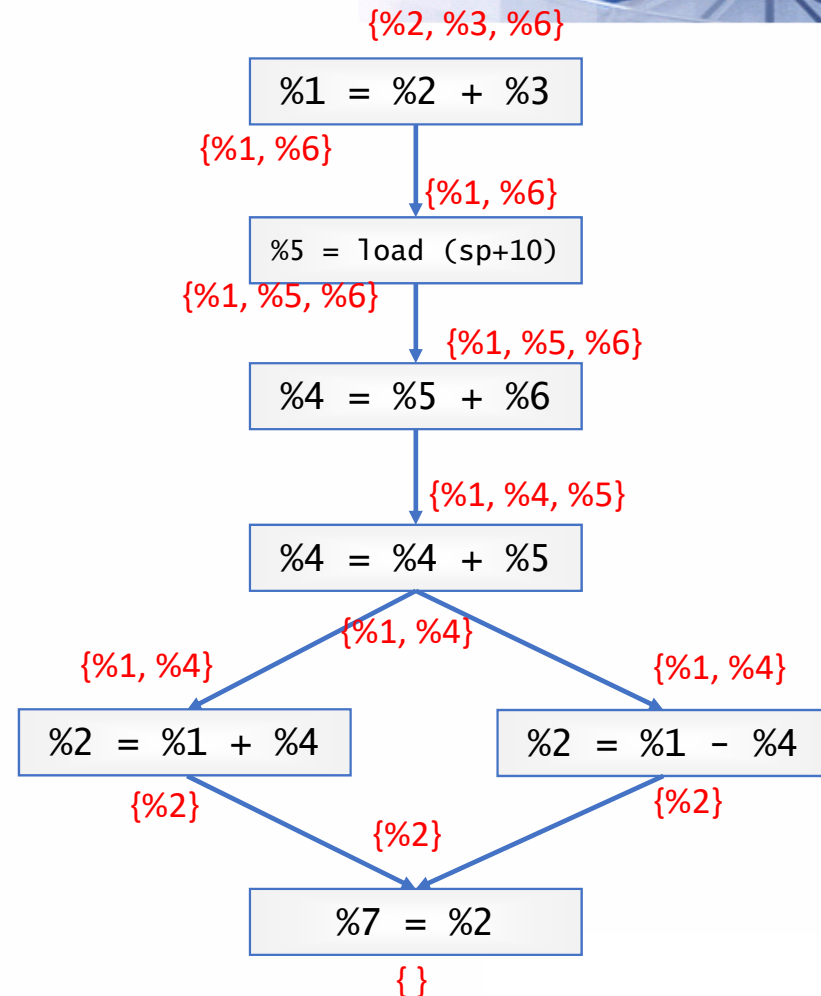
- If we have only 3 physical registers, it is impossible to assign a color to %5.
- In this case, %5 is spilled to a stack location.
- We rewrite the program by adding loads and stores for %5.
- Try to color the new program.



Register Allocation via Graph Coloring

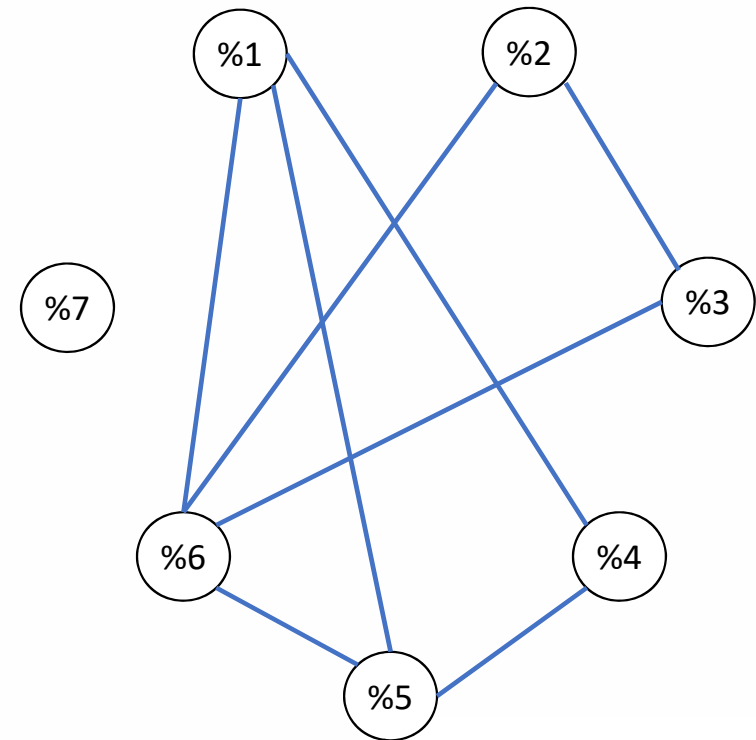
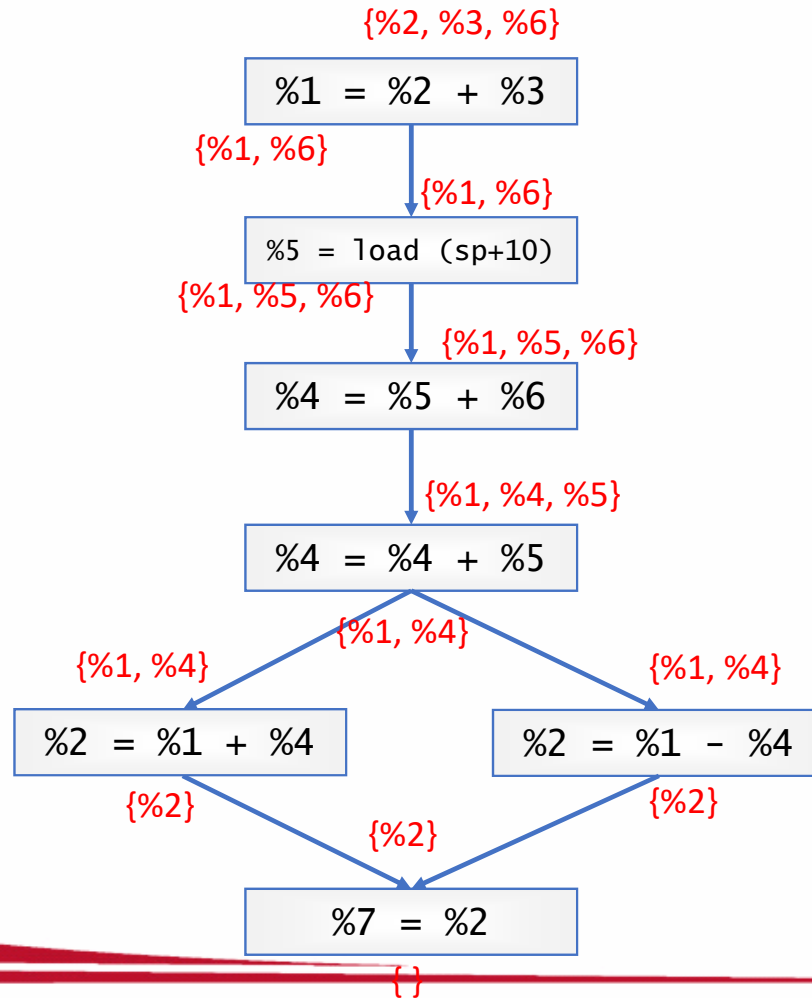
- **Spill**

- If we have only 3 physical registers, it is impossible to assign a color to %5.
- In this case, %5 is spilled to a stack location.
- We rewrite the program by adding loads and stores for %5.
- Try to color the new program.



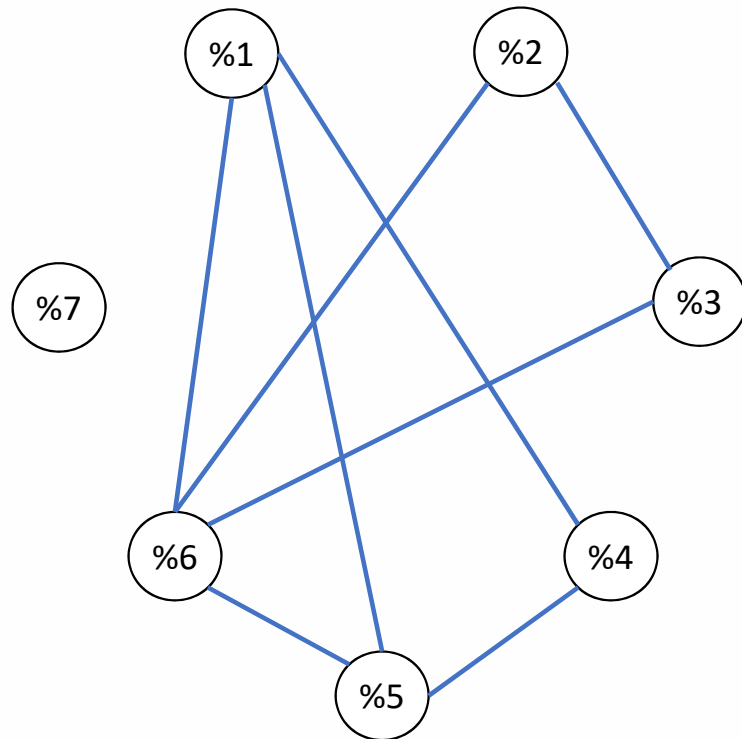
Register Allocation via Graph Coloring

- Spill

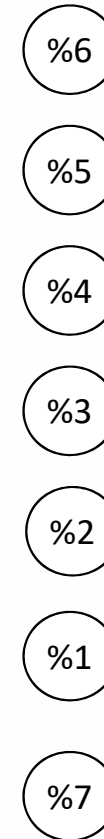


Register Allocation via Graph Coloring

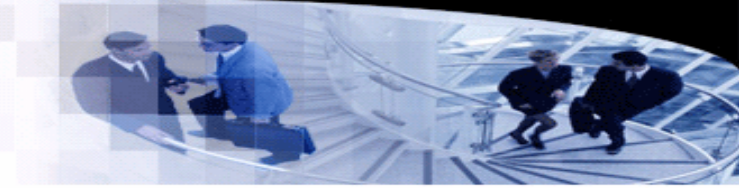
- Spill



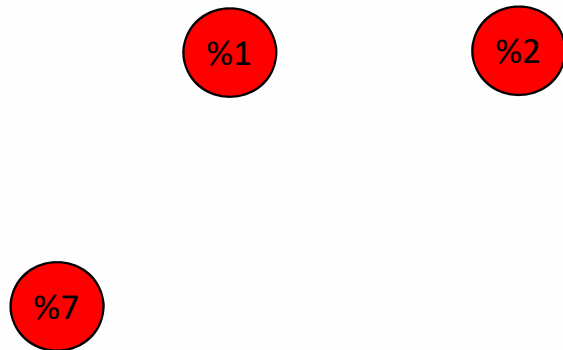
Removing order



Register Allocation via Graph Coloring



- Spill

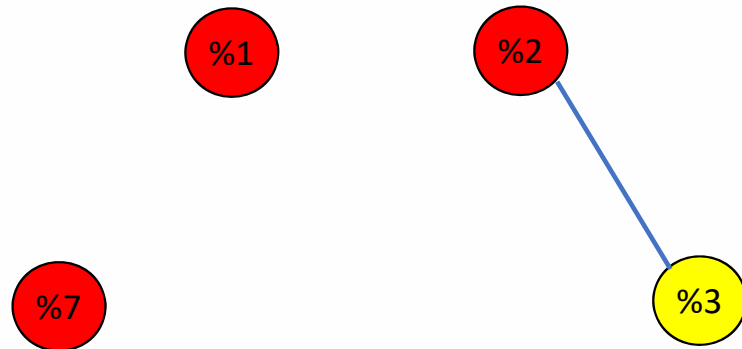


Removing order



Register Allocation via Graph Coloring

- Spill

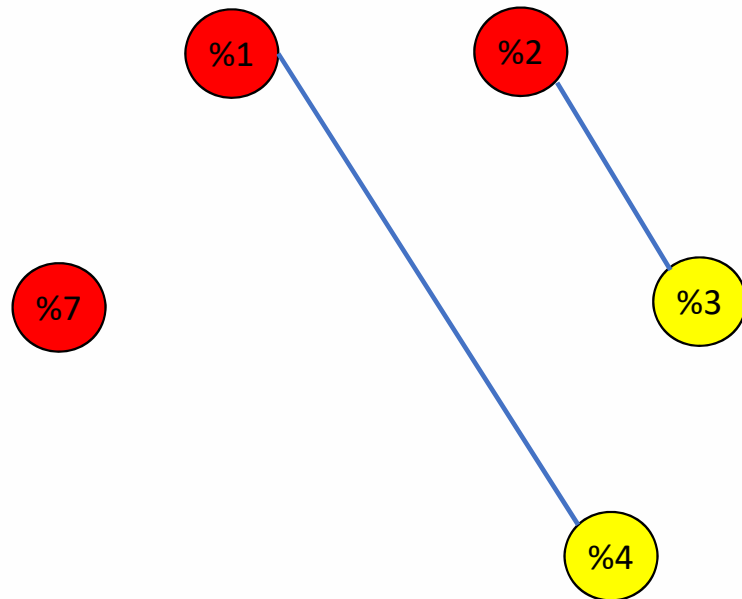


Removing order



Register Allocation via Graph Coloring

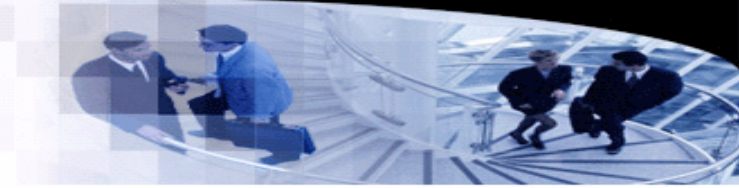
- Spill



Removing order

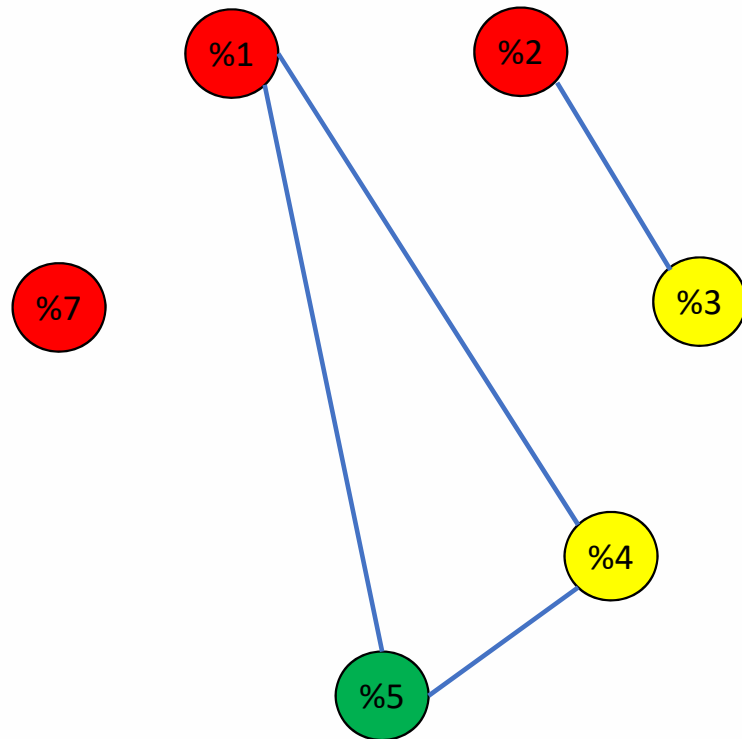
%6

%5

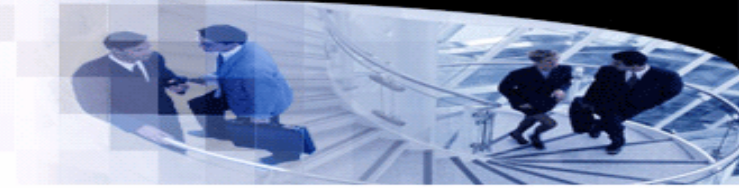


Register Allocation via Graph Coloring

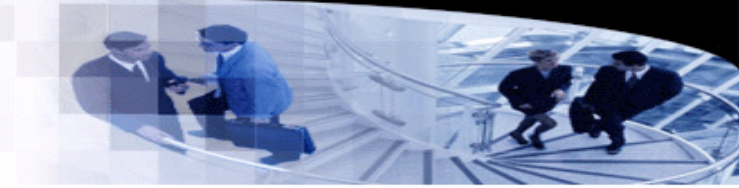
- Spill



Removing order



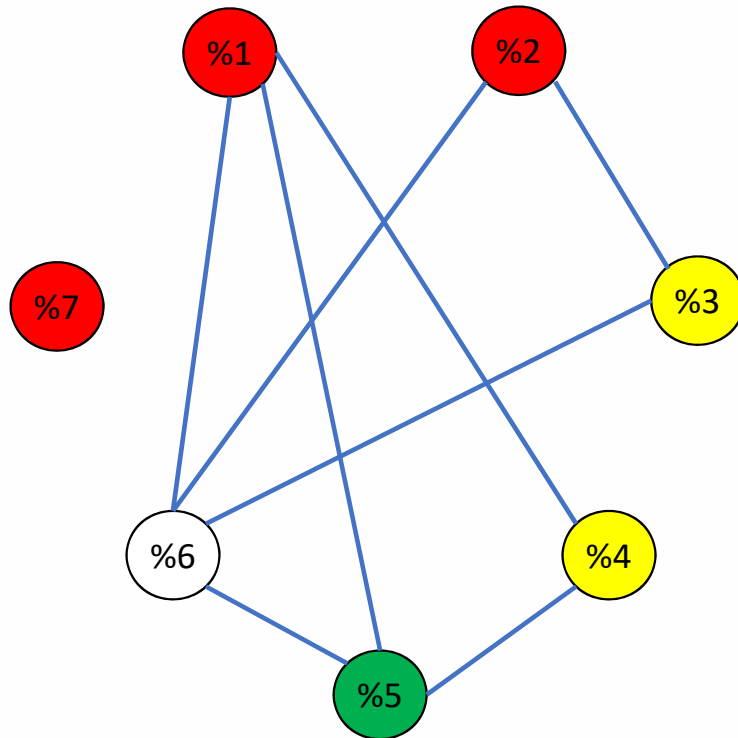
Register Allocation via Graph Coloring



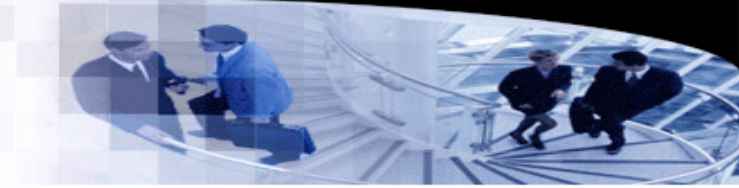
- Spill

- Not 3-colorable.

Removing order

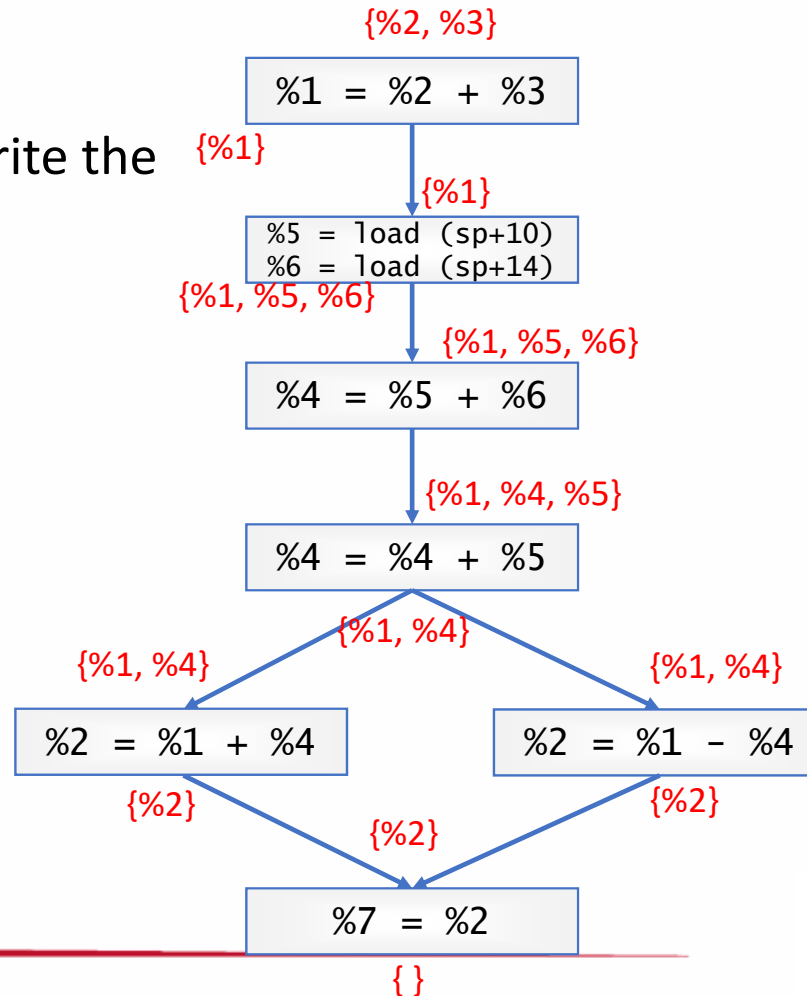


Register Allocation via Graph Coloring



- Spill

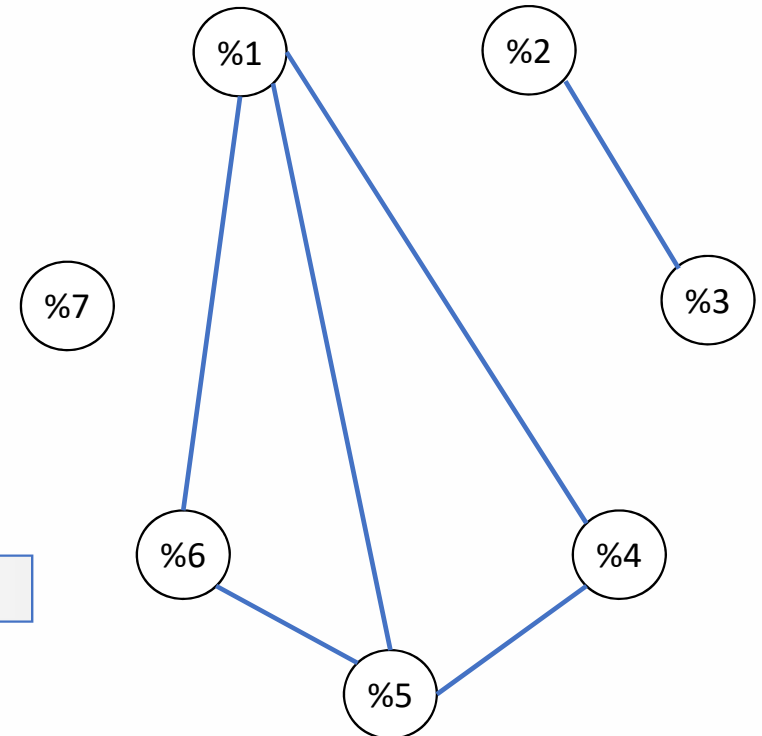
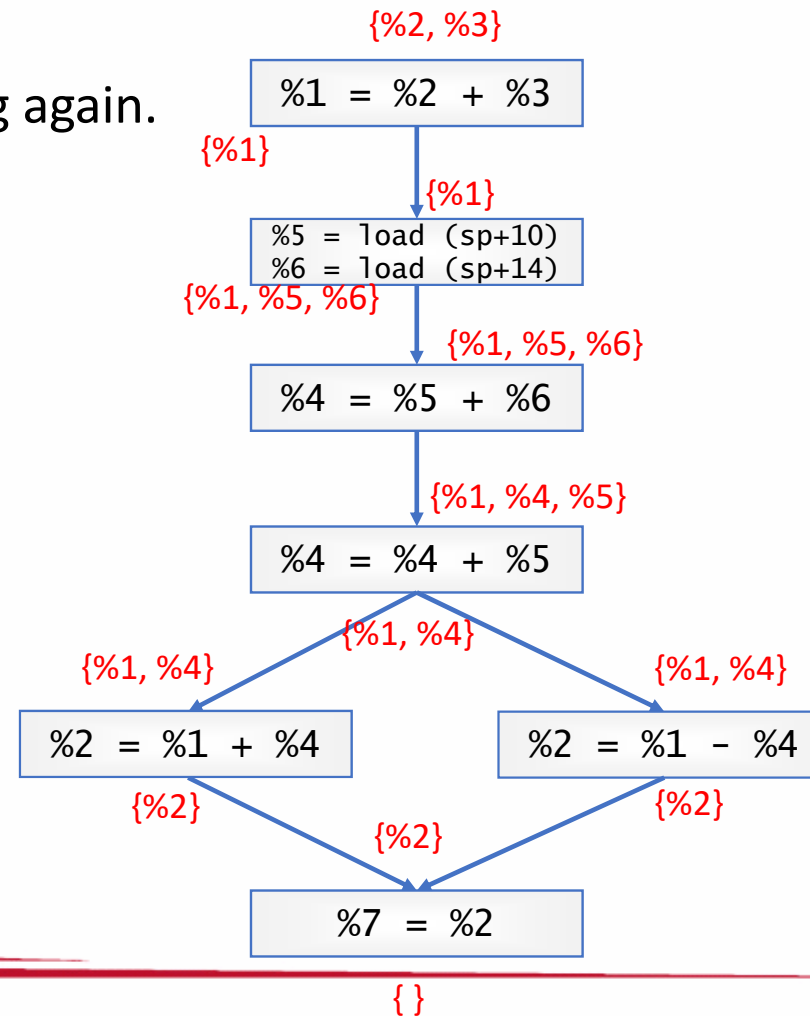
- Not 3-colorable.
- We have to spill %6 and rewrite the program again!



Register Allocation via Graph Coloring

- Spill

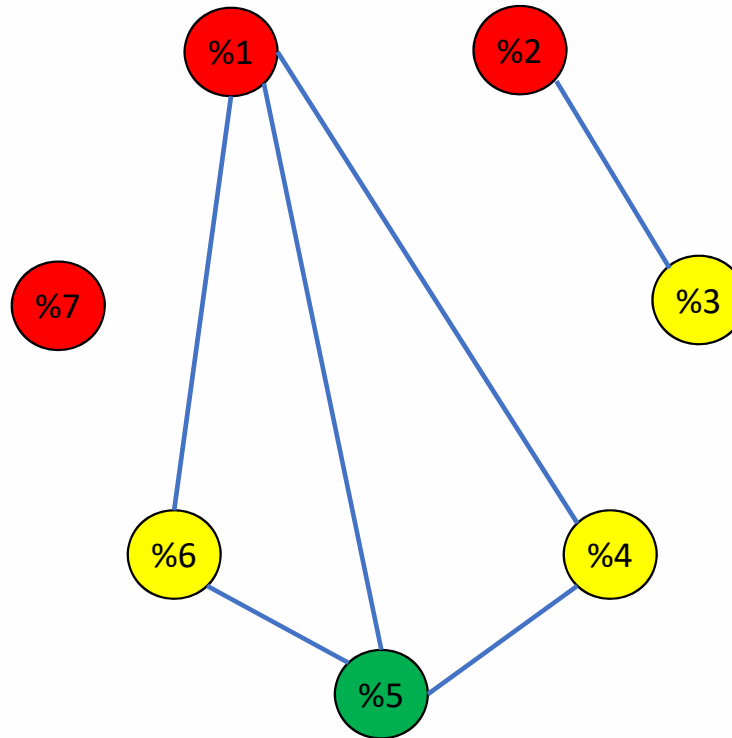
- Try graph coloring again.



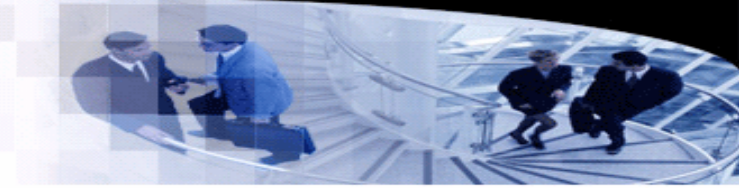
Register Allocation via Graph Coloring

- Spill

- Try graph coloring again.



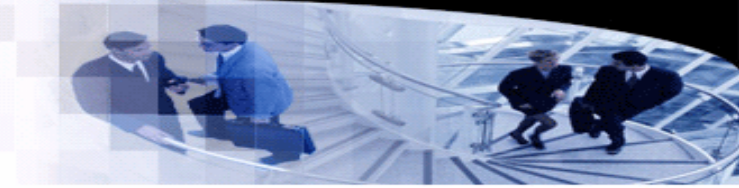
Register Allocation via Graph Coloring



- **Algorithm**

- 1. Build the interference graph G via liveness analysis.
- 2. Try to color the interference graph.
- 3. The algorithm terminates if G is k -colorable.
- 4. Spill a vertex whose number of edges is larger than k .
 - The selection of a vertex to spill is also heuristic.
- 5. Rewrite the program by adding loads and stores.
- 6. Goto 1.

Register Allocation via Graph Coloring



- Other potential improvements

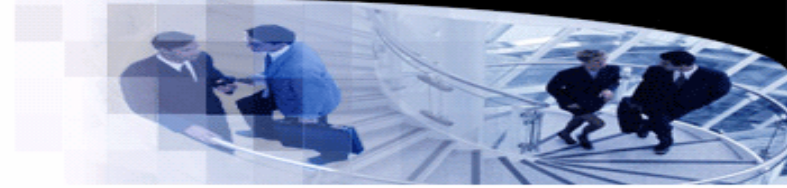
- Coalescing

- Some optimizations may increase live ranges that cause register pressure.
 - For example, copy propagation and common subexpression elimination.
 - It can be avoid by merging vertices in the interference graph.
 - Given a statement $a = b$, we can coalesce a and b in the graph if a and b do not interfere each other.
 - When a and b are assigned the same register t_i , the original statement can be rewritten as $t_i = t_i$.
 - This redundant statement will be removed by other optimizations.

- Risks

- Coalescing may make a k -colorable graph uncolorable.

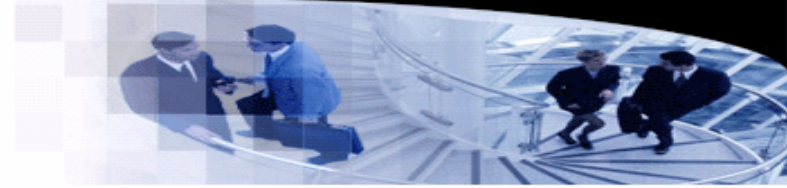
Linear Scan



- Idea

- A greedy algorithm proposed by Poletto & Sarkar.
- <https://dl.acm.org/doi/abs/10.1145/330249.330250>
- Use live interval to express the liveness of variable.
- Scan the linear IR from the first statement to the last statement.
- When a live interval begins at the current scan point, give a free register to the variable.
- When a live interval ends at the current scan point, free the register assigned to the variable.
- However, spilling variables is still inevitable.

Linear Scan



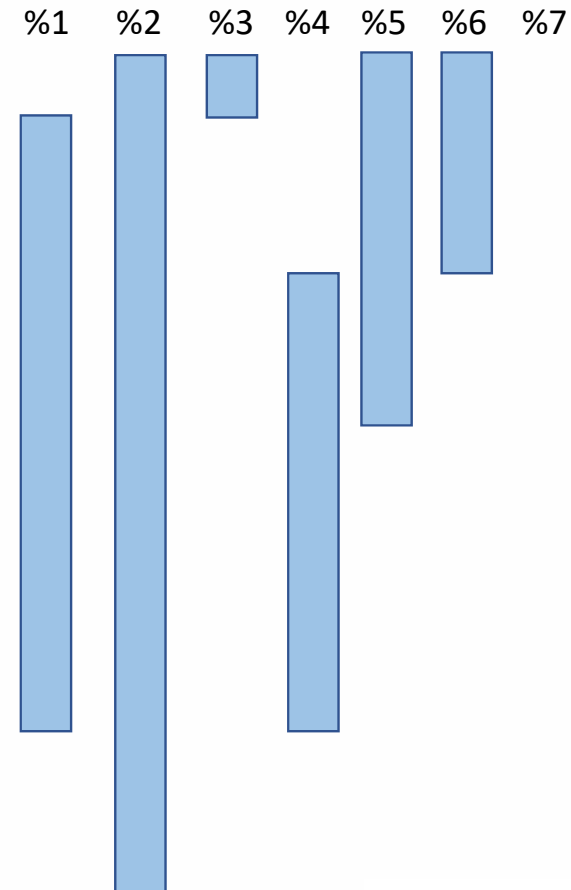
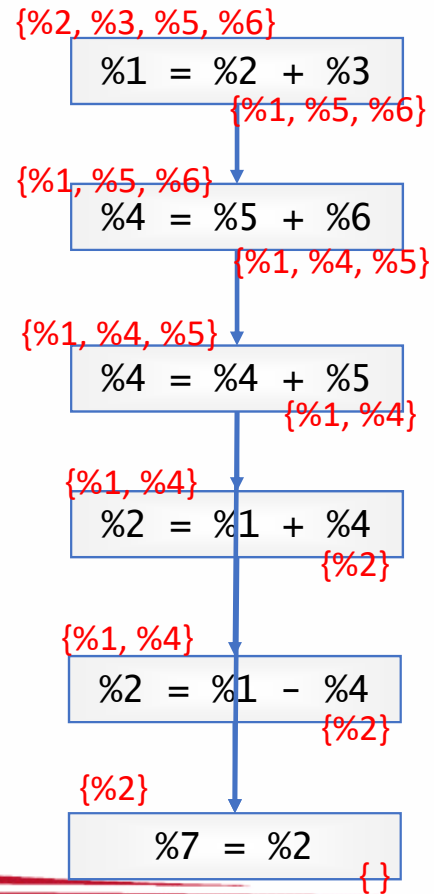
- **Live intervals**

- The **live range** of a variable is the set of program locations where the variable is live.
- Given a sequential IR of program, the **live interval** of a variable is the smallest subrange of IR containing all alive ranges.
- Live intervals are less precise compared to live ranges.
- It is more suitable for register allocation.

Linear Scan

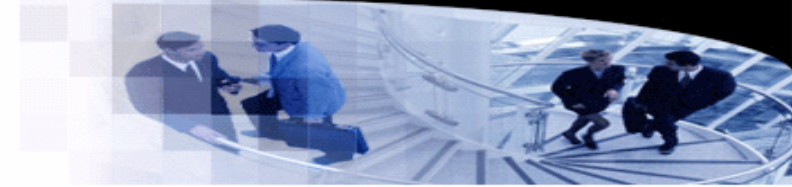
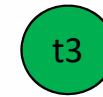
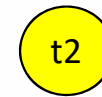
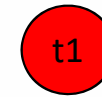
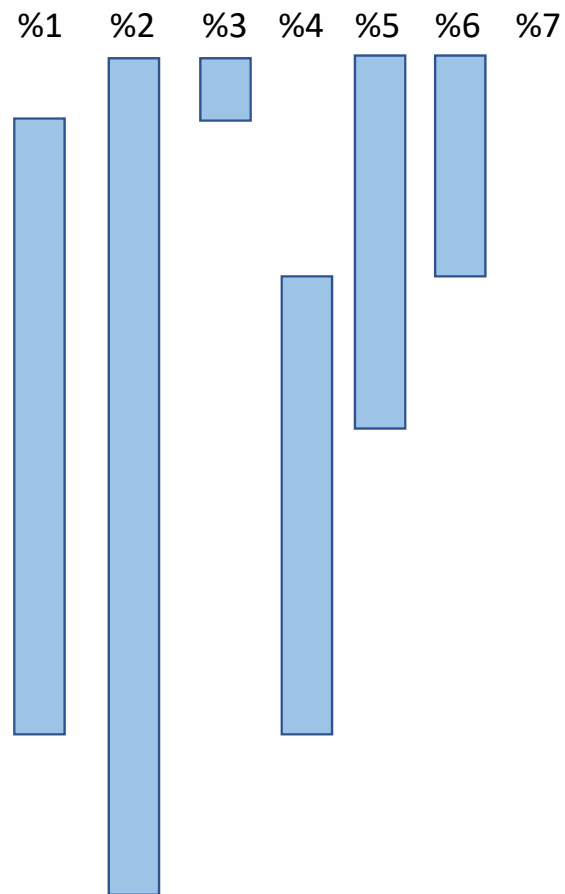
- Live intervals

- Example



Linear Scan

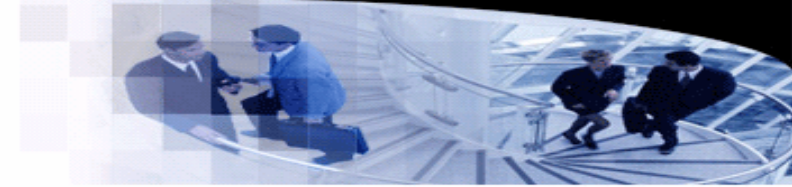
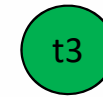
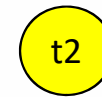
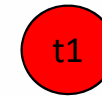
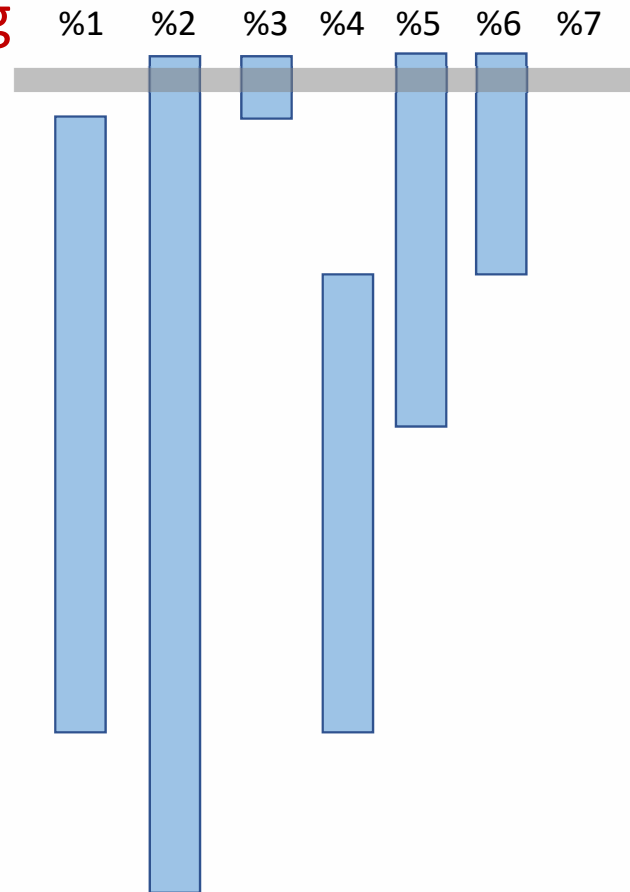
- Allocation via scanning
 - 4 physical registers



Linear Scan

- Allocation via scanning

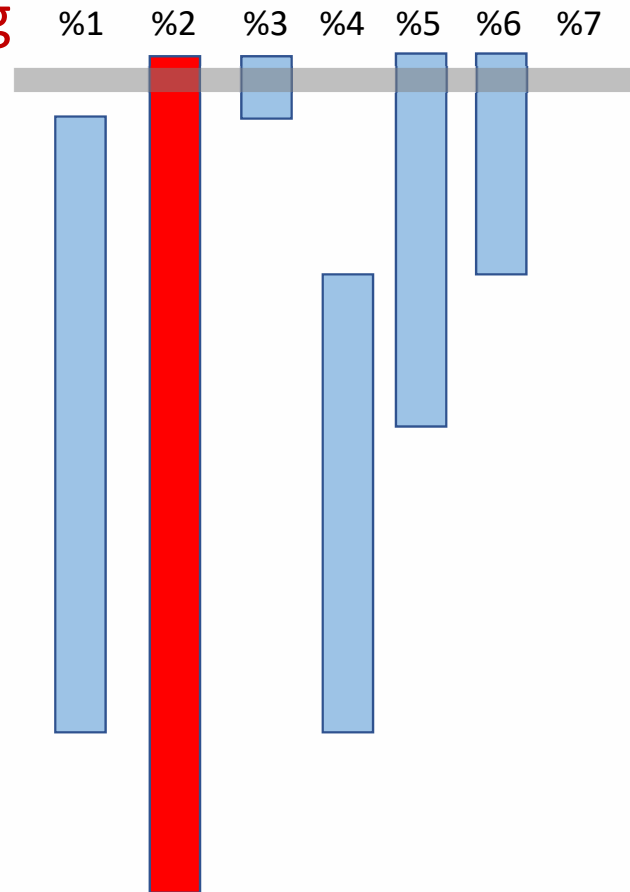
- 4 physical registers



Linear Scan

- Allocation via scanning

- 4 physical registers
- Assign t1 to %2.

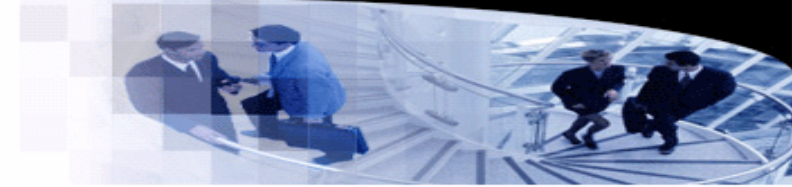


t1

t2

t3

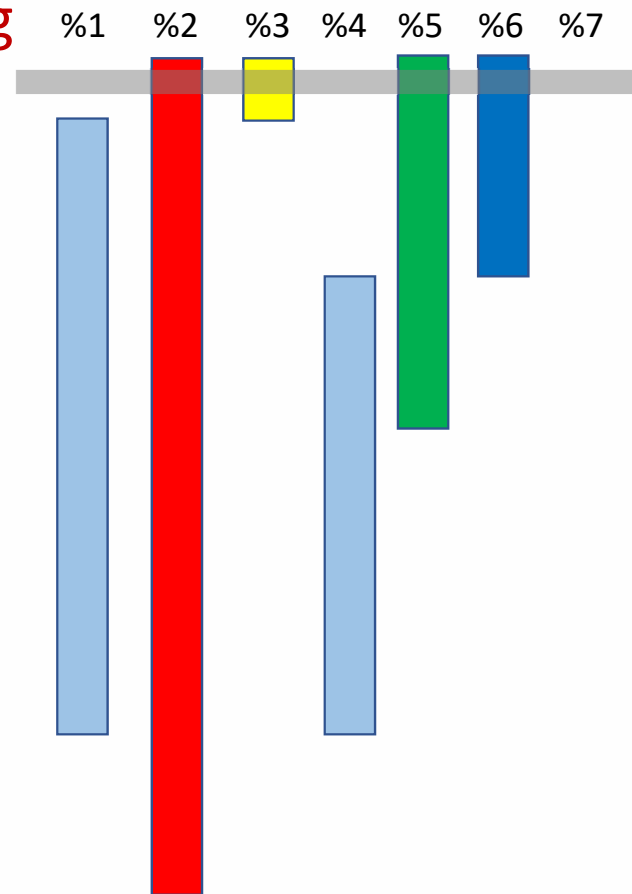
t4



Linear Scan

- Allocation via scanning

- 4 physical registers
- Assign t1 to %2.
- t2 → %3.
- t3 → %5.
- t4 → %6.

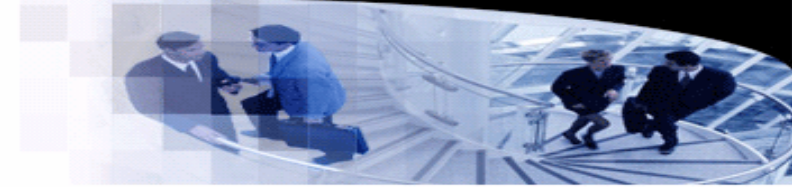


t1

t2

t3

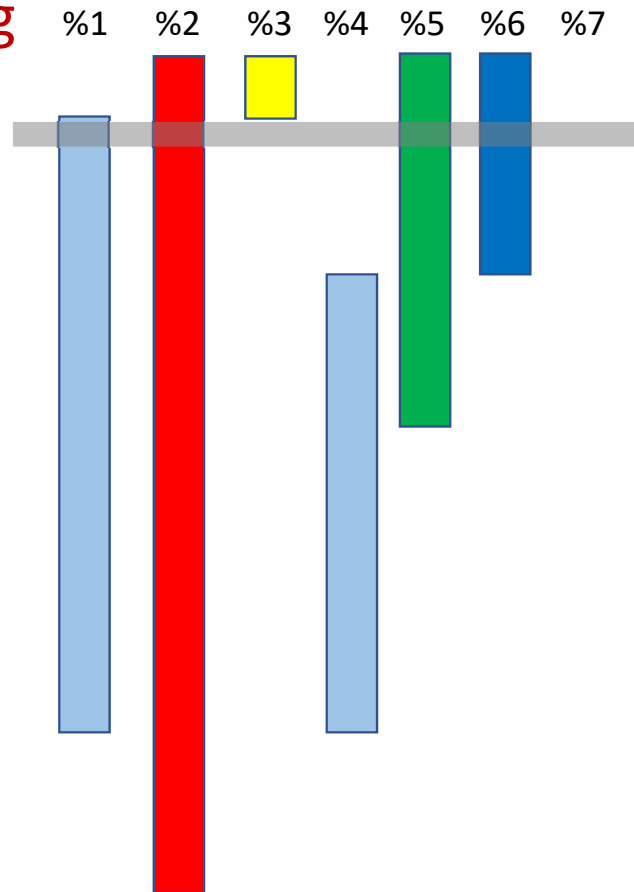
t4



Linear Scan

- Allocation via scanning

- 4 physical registers
- Assign t1 to %2.
- t2 → %3.
- t3 → %5.
- t4 → %6.
- Free t2.

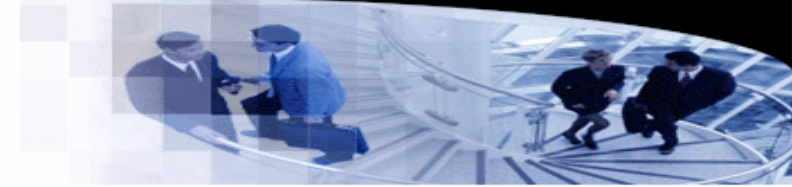


t1

t2

t3

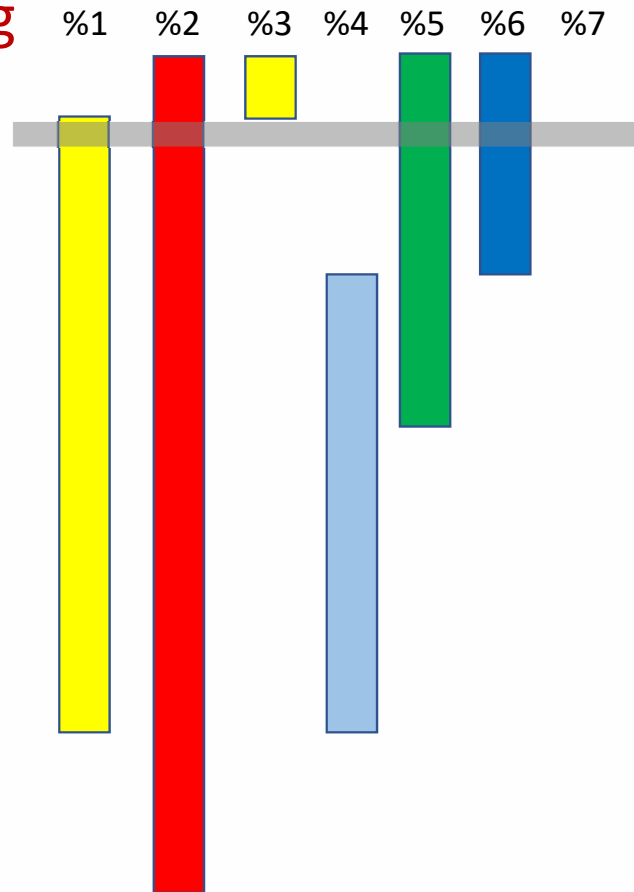
t4



Linear Scan

- Allocation via scanning

- 4 physical registers
- Assign t1 to %2.
- $t2 \rightarrow \%3$.
- $t3 \rightarrow \%5$.
- $t4 \rightarrow \%6$.
- Free t2.
- $t2 \rightarrow \%1$.

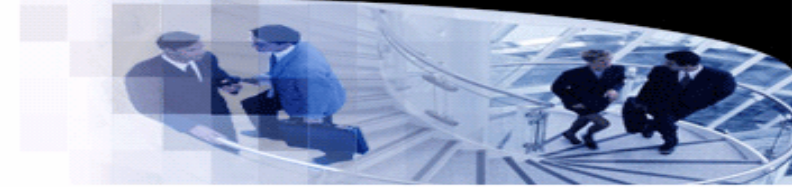


t1

t2

t3

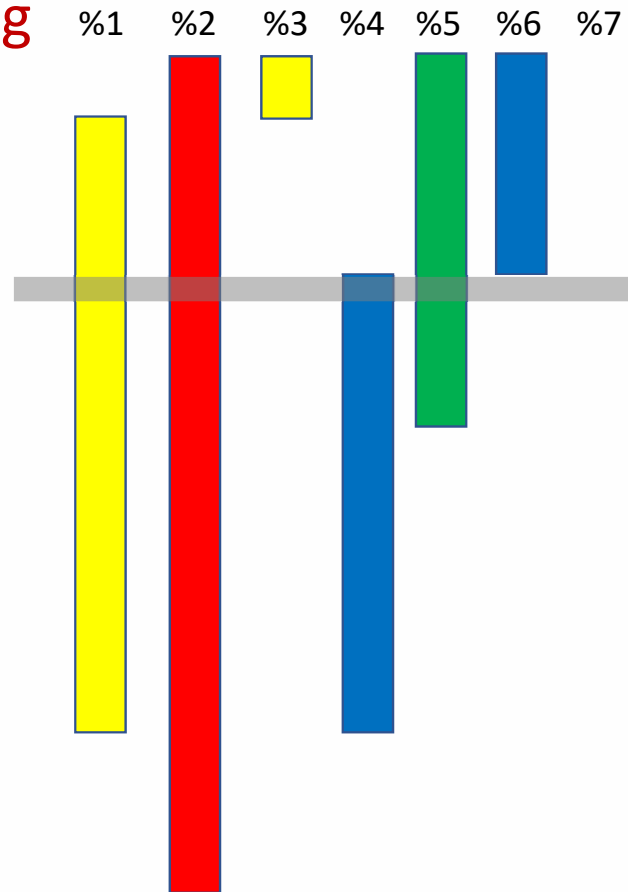
t4



Linear Scan

- Allocation via scanning

- 4 physical registers
- Assign t1 to %2.
- t2 → %3.
- t3 → %5.
- t4 → %6.
- Free t2.
- t2 → %1.
- Free t4.
- t4 → %4.

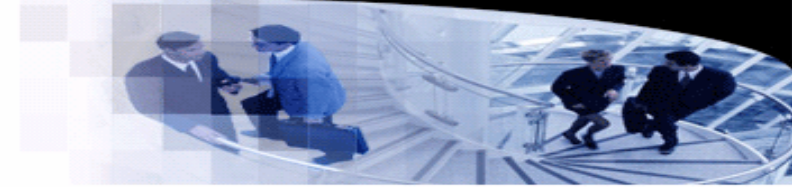


t1

t2

t3

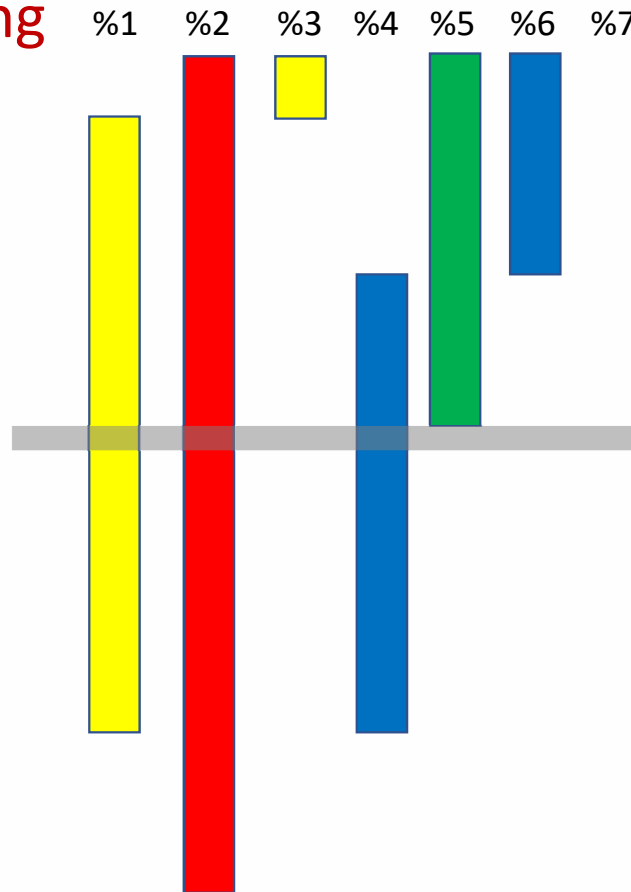
t4



Linear Scan

- Allocation via scanning

- 4 physical registers
- Assign t1 to %2.
- $t2 \rightarrow \%3$.
- $t3 \rightarrow \%5$.
- $t4 \rightarrow \%6$.
- Free t2.
- $t2 \rightarrow \%1$.
- Free t4.
- $t4 \rightarrow \%4$.
- Free t3.



t1

t2

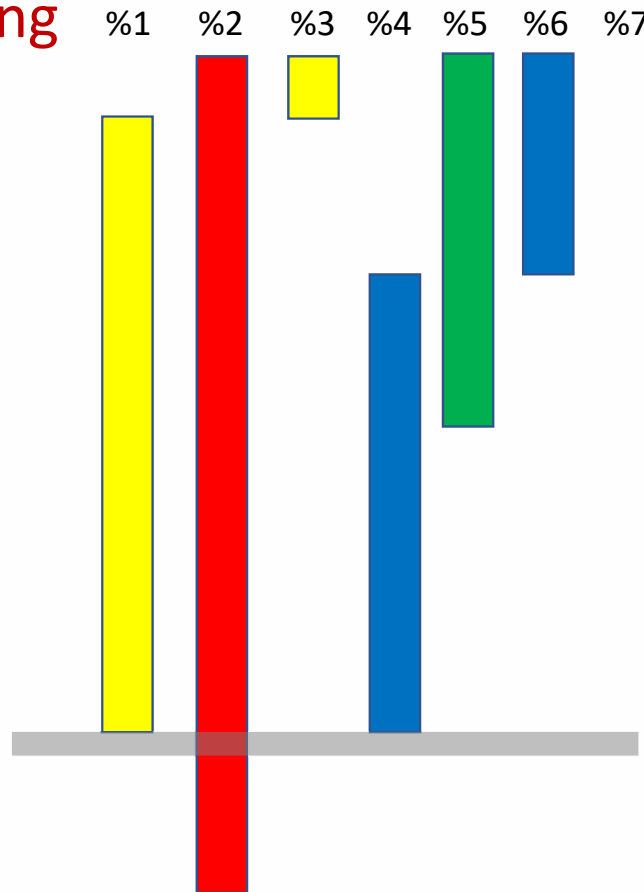
t3

t4

Linear Scan

- Allocation via scanning

- 4 physical registers
- Assign t1 to %2.
- $t2 \rightarrow \%3$.
- $t3 \rightarrow \%5$.
- $t4 \rightarrow \%6$.
- Free t2.
- $t2 \rightarrow \%1$.
- Free t4.
- $t4 \rightarrow \%4$.
- Free t3.
- Free t2 and t4.

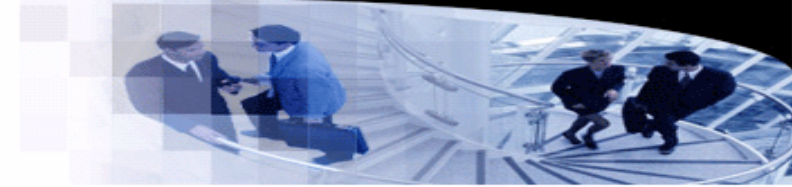


t1

t2

t3

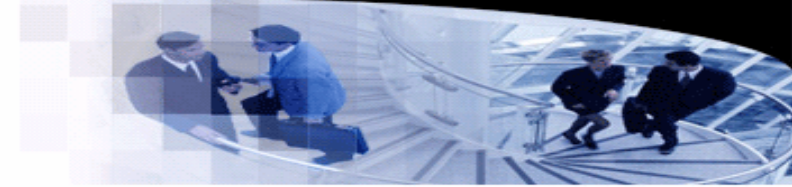
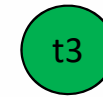
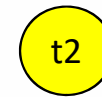
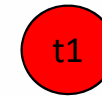
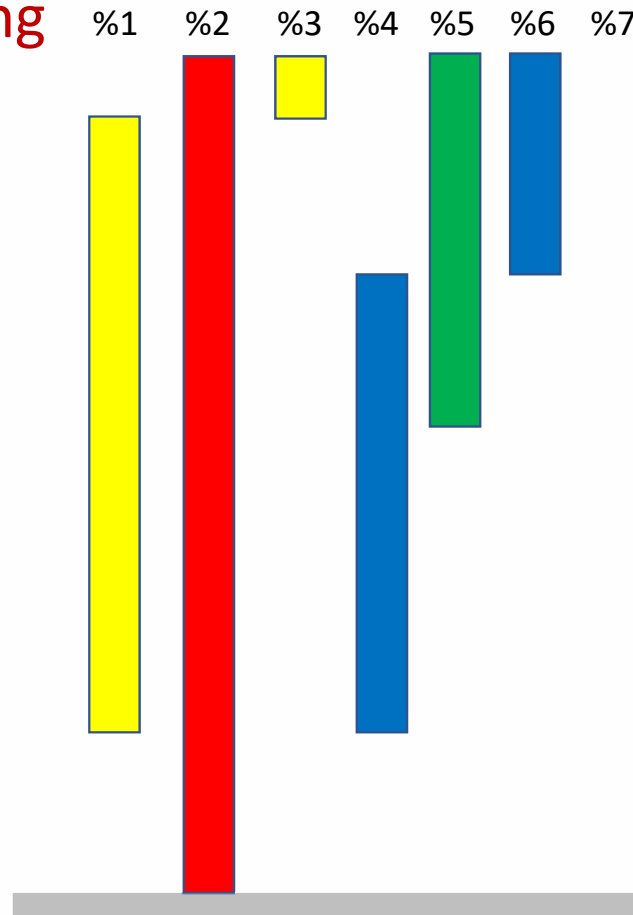
t4



Linear Scan

- Allocation via scanning

- 4 physical registers
- Assign t1 to %2.
- $t2 \rightarrow \%3$.
- $t3 \rightarrow \%5$.
- $t4 \rightarrow \%6$.
- Free t2.
- $t2 \rightarrow \%1$.
- Free t4.
- $t4 \rightarrow \%4$.
- Free t3.
- Free t2 and t4.
- Free t1.



Comparison of Performance

- Graph coloring vs linear scan

